

A PRACTICAL HANDBOOK OF

MACHINE LEARNING

From Data to Deployment: Concepts, Techniques
and Real-World Implementation



Data Understanding



Data Cleaning &
Preprocessing



Feature Engineering



Model Selection



Model Evaluation



Deployment



Advanced Concepts



AUTHOR

Abhishek R. Kayasth

Preface

This handbook is a structured and practical guide to understanding the complete workflow of Machine Learning, designed for learners who already possess a basic foundation in programming and introductory machine learning concepts. It is not intended for absolute beginners, but rather for individuals who want to strengthen their understanding of how machine learning is applied in real-world scenarios.

The content of this handbook follows the **end-to-end Machine Learning lifecycle**, beginning with Problem Understanding and progressing through Data Understanding (EDA), Data Cleaning, Feature Engineering, Model Selection, Model Evaluation, and Deployment. Each stage is explained with a focus on practical implementation, commonly used techniques, and real-world relevance.

In addition to core concepts, this handbook also covers important advanced and supporting topics such as **Hyperparameter Tuning, Cross Validation, Dimensionality Reduction (PCA), Learning Curves, Handling Imbalanced Data, Pipelines, Model Monitoring, A/B Testing, Regularization, and Recommendation Systems**. These topics are essential for building robust and production-ready machine learning systems.

A key highlight of this work is that **every major concept included in this handbook is supported by practical implementations**. All code examples, experiments, and hands-on practice related to these topics are available on GitHub: https://github.com/Abhixtech4/ML_practice

This allows readers not only to understand concepts theoretically but also to see how they are applied in real projects.

This handbook is intended to serve as a **quick reference, revision guide, and practical companion** for students, aspiring data scientists, and machine learning practitioners.

This document has been compiled and written by

Abhishek R. Kayasth,

With the aim of organizing essential machine learning knowledge into a clear, structured, and practical format.

How to Use This Handbook

This handbook is designed as a **practical guide and reference** for understanding and applying machine learning concepts in a structured way. It is best used alongside hands-on practice rather than as a purely theoretical resource.

1. Follow the Machine Learning Lifecycle

The handbook is organized according to the real-world **Machine Learning lifecycle**:

Problem Understanding → Data Understanding → Data Cleaning → Feature Engineering → Model Selection → Model Evaluation → Deployment

It is recommended to follow this sequence, especially if you are working on a project, as each step builds on the previous one.

2. Learn Concept → Apply Practically

Each topic in this handbook is intentionally concise and focused on **what is important in practice**. After reading a section:

- Try implementing the concept yourself
- Experiment with different datasets
- Modify parameters and observe results

All practical implementations related to this handbook are available on GitHub:

https://github.com/Abhixtech4/ML_practice

This will help you bridge the gap between theory and real-world application.

3. Use as a Revision Guide

This handbook is also useful as a **quick revision resource**. Before interviews, projects, or competitions, you can:

- Review key concepts quickly
- Recall important techniques and methods
- Refresh your understanding of workflows

4. Focus on Key Decision Points

Machine learning is not just about code—it's about making the right decisions. While using this handbook, pay attention to:

- When to use which model
- How to handle missing data or outliers

- Which evaluation metric to choose
- When to apply techniques like scaling, encoding, or PCA

Understanding these decisions is more important than memorizing code.

5. Explore Advanced Topics Gradually

The later chapters include more advanced concepts such as:

- Hyperparameter tuning
- Cross validation
- Pipelines
- Model monitoring

You do not need to master everything at once. Learn them step by step and revisit them as you gain more experience.

6. Build Alongside

The best way to use this handbook is to **build projects alongside it**. Apply each section to a real dataset and try to complete the full pipeline from start to deployment.

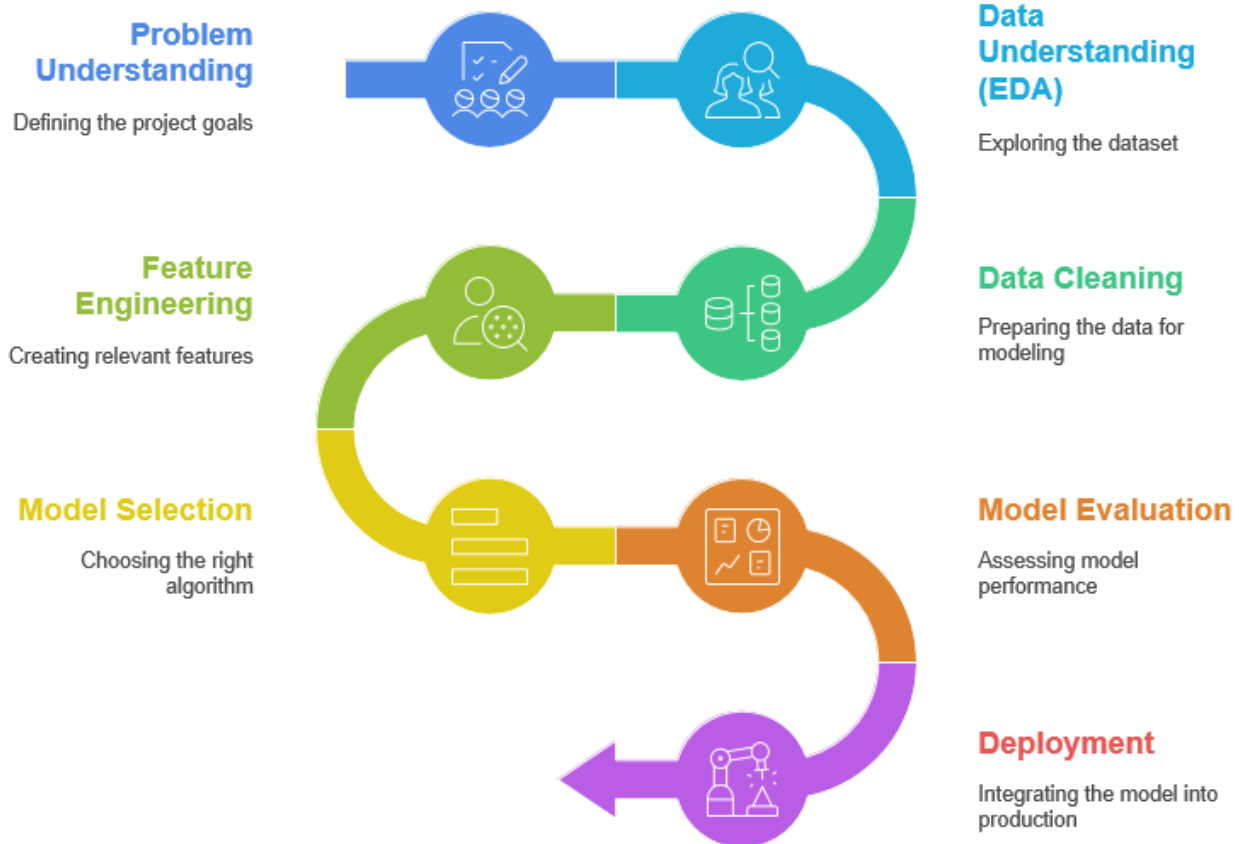
This handbook is meant to be a **learning companion**, not just a reading document. The more you practice and experiment, the more value you will gain from it.

Index

Chapter. no	Table of content	Page no.
	Machine learning life Cycle	1
1	Problem Understanding	3
2	Data Understanding	4
2.1	Basic Overview	4
2.2	Target Variable Analysis	5
2.3	Feature Understanding	6
3	Data Cleaning	7
3.1	Missing Values	7
3.2	Duplicates	8
3.3	Outliers	9
3.4	Data Type Correction	10
3.5	Inconsistent Values	11
4	Feature Engineering	12
4.1	Encoding Categorical Variables	12
4.2	Feature Scaling	14
4.3	Handling Skewness	15
4.4	Feature Creation	15
4.5	Feature Selection	16
4.6	Train-Test Split	18
5	Model Selection	20
5.1	Models for Structured data	21
5.1.1	Supervised machine learning model	21
5.1.1.1	Regression models	21
5.1.1.2	Classification models	22
5.1.1.3	Models used for both Classification & Regression	23
5.1.2	Unsupervised learning	24
5.2	Models for unstructured data	25
5.2.1	Natural Language Processing	26
5.2.2	TF-IDF + Machine learning	26
5.2.3	Transformers / Deep learning	26
5.2.4	Image data	26
5.2.5	Audio	27
6	Model evaluation	28
6.1	Hyperparameter Tuning	28
6.2	Deep Evaluation	30
6.3	ROC Curve	31
6.4	Precision–Recall Curve	32
6.5	Threshold Tuning	32
6.6	Feature Importance	33
6.7	SHAP Values (Advanced)	34
7	Deployment	35
7.1	Save Model, Scaler, and Encoder	35
7.2	Build API Using FastAPI	36

8	Important Concept	38
8.1	Dimensionality Reduction	38
8.2	Learning Curves	39
8.3	Handling Imbalanced Data	40
8.4	Pipelines	41
8.5	Model Persistence Versioning	42
8.6	Model Monitoring	42
8.7	A/B Testing	43
8.8	Regularization	44
8.9	Cross Validation	44
8.9.1	K-Fold cross validation	45
8.9.2	Stratified K-Fold	46
8.10	Recommendation Systems	46

Machine learning life Cycle:



1. Problem Understanding:

Problem understanding is the process of clearly defining the objective of a machine learning project. It involves identifying whether the task is classification, regression, clustering, or another type of problem, determining the target variable, and understanding constraints such as business goals, performance metrics, and available resources. A well-defined problem ensures alignment between technical implementation and real-world requirements.

2. Data Understanding (EDA):

Exploratory Data Analysis (EDA) involves examining the dataset to understand its structure, patterns, and relationships. It includes analyzing data types, distributions, summary statistics, correlations, missing values, and outliers. Visualization techniques such as histograms, boxplots, and heatmaps are commonly used to uncover trends and anomalies that influence model development.

3. Data Cleaning:

Data cleaning focuses on preparing raw data for modeling by handling missing values, removing duplicates, correcting inconsistencies, and treating outliers. It may also include formatting variables, resolving data entry errors, and ensuring consistency in categorical labels. Clean data improves model reliability and accuracy.

4. Feature Engineering:

Feature engineering involves transforming raw data into meaningful features that enhance model performance. This includes encoding categorical variables, scaling numerical features, creating interaction terms, extracting time-based components, and generating new variables from existing ones. Effective feature engineering improves predictive power.

5. Model Selection:

Model selection is the process of choosing appropriate algorithms based on the problem type and data characteristics. Different models such as linear regression, decision trees, support vector machines, or neural networks are compared to identify the most suitable approach for the task.

6. Model Evaluation:

Model evaluation assesses how well a trained model performs on unseen data. Metrics such as accuracy, precision, recall, F1-score, mean squared error, or ROC-AUC are used depending on the problem type. Validation techniques like train-test split or cross-validation ensure generalization capability.

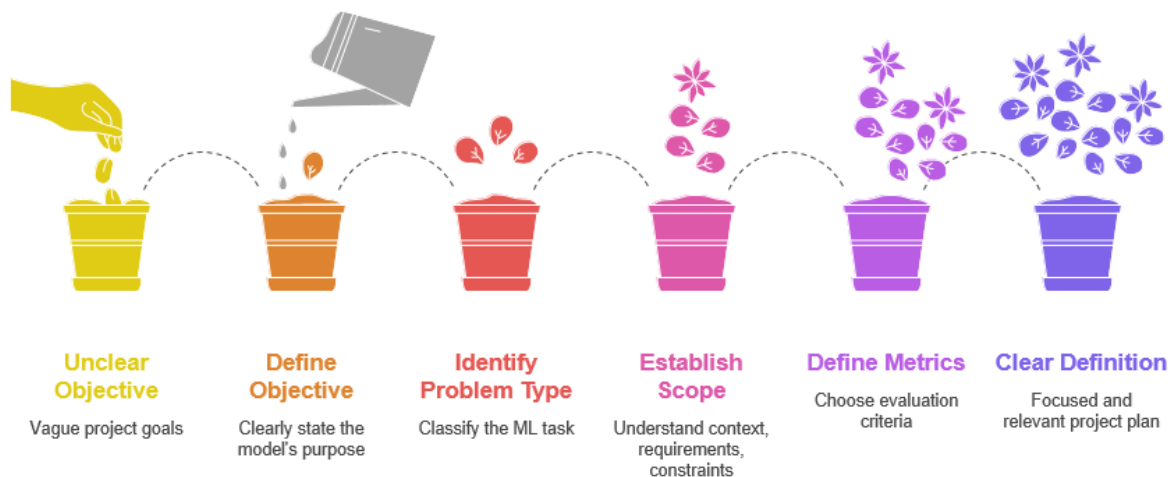
7. Deployment:

Deployment involves integrating the trained model into a production environment where it can generate predictions in real time or batch mode. This may include exposing the model through APIs, embedding it into applications, or automating prediction pipelines. Continuous monitoring ensures consistent performance over time.

Ch-1: Problem Understanding

Problem understanding is the initial and most critical stage of a machine learning project. It involves clearly defining the objective that the model is expected to achieve. This includes identifying the type of problem—such as classification, regression, clustering, or recommendation—and determining the target variable to be predicted.

At this stage, the scope of the project is established by understanding the domain context, business requirements, constraints, and success criteria. Defining appropriate evaluation metrics aligned with the objective is also part of this process. A clear problem definition ensures that data collection, modeling decisions, and evaluation methods remain focused and relevant to the intended outcome.



Before touching data:

Ask:

- Is this classification, regression, clustering and recommendation?
- What is the business objective?
- What metric matters? (Accuracy? RMSE? Revenue?)

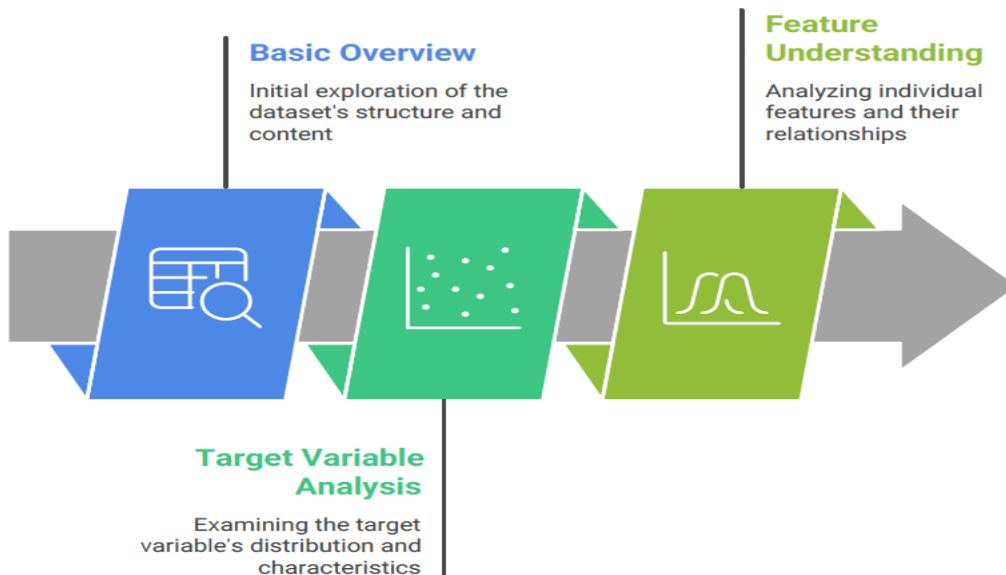
Example:

- Predict house price → Regression
- Predict fraud or not → Classification
- Group customers → Clustering

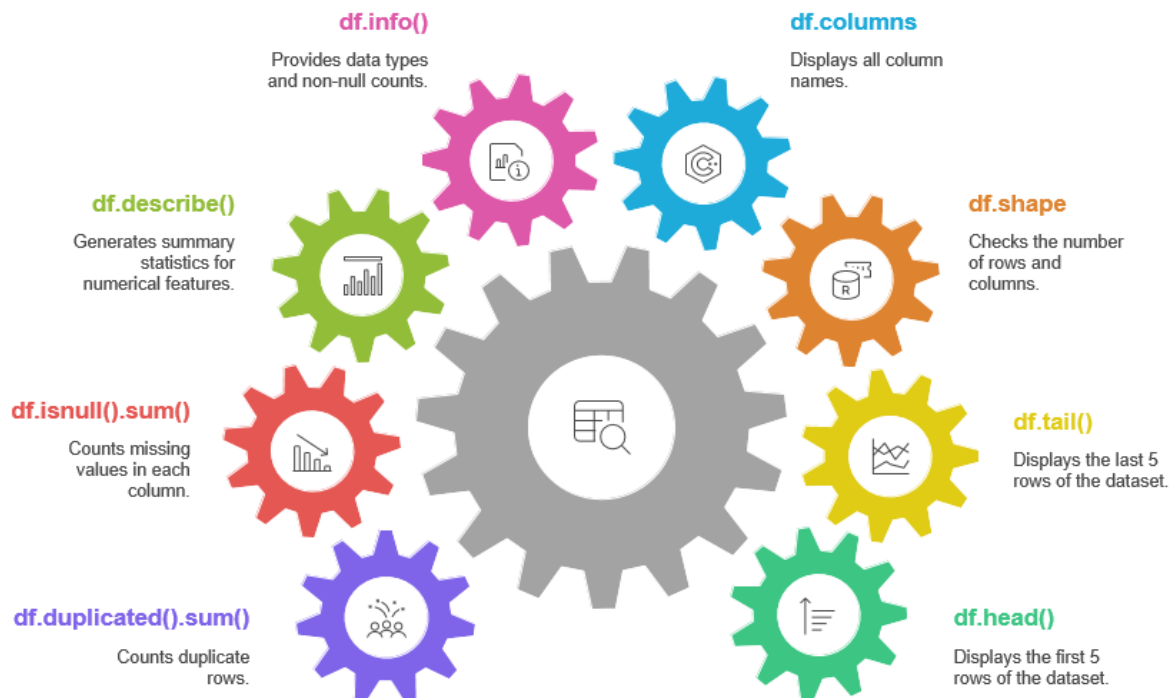
Ch-2: Data Understanding

Data Understanding is the step where we explore the dataset to understand its structure, content, and characteristics before applying preprocessing or modeling.

Data Understanding Process



Ch-2.1: Basic Overview



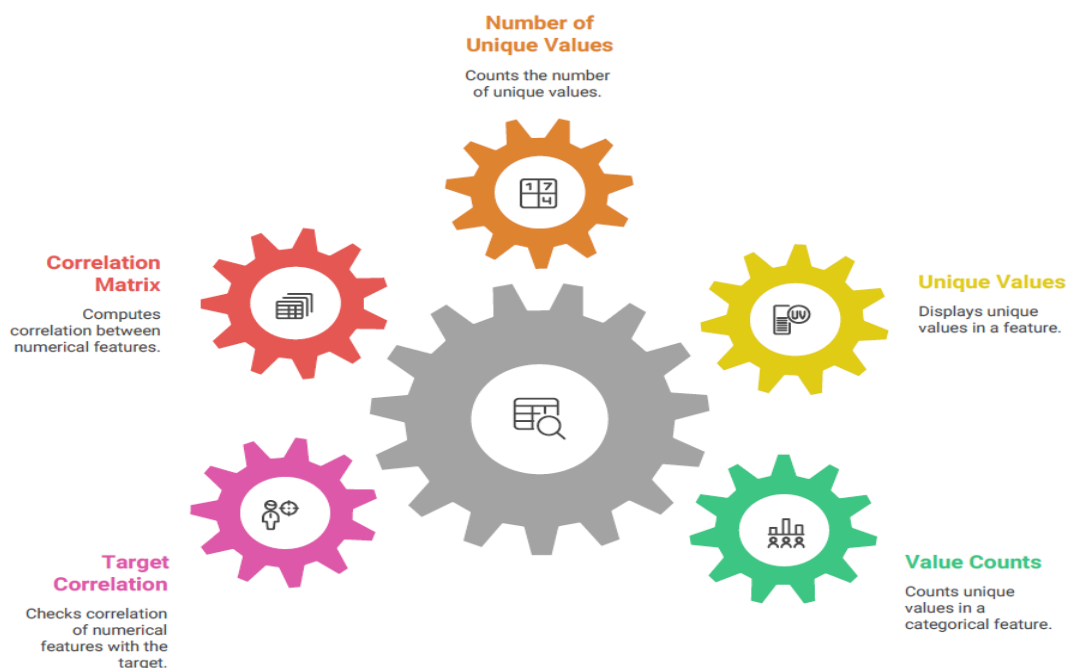
1. The very first thing to do before starting project is to check whether the source of dataset is trustworthy.
2. **df.head()** :
Used to display the first 5 rows of the dataset.
3. **df.tail()** :
Used to display the last 5 rows of the dataset.
4. **df.shape** :
Used to check the number of rows and columns in the dataset.
5. **df.columns** :
Used to display all column names.
6. **df.info()** :
Used to get information about data types and non-null values.
7. **df.describe()** :
Used to generate summary statistics (mean, std, min, max, etc.) for numerical features.
8. **df.isnull().sum()** :
Used to check the number of missing values in each column.
9. **df.duplicated().sum()** :
Used to check the number of duplicate rows.

Ch-2.2: Target Variable Analysis



1. **df["target"].value_counts()** :
Used to count occurrences of each class in a classification problem.
2. **df["target"].value_counts(normalize=True)** :
Used to calculate class distribution in percentage form.
3. **df["target"].describe()** :
Used to view summary statistics of the target variable in regression.
4. **df["target"].skew()** :
Used to check skewness of the target variable.

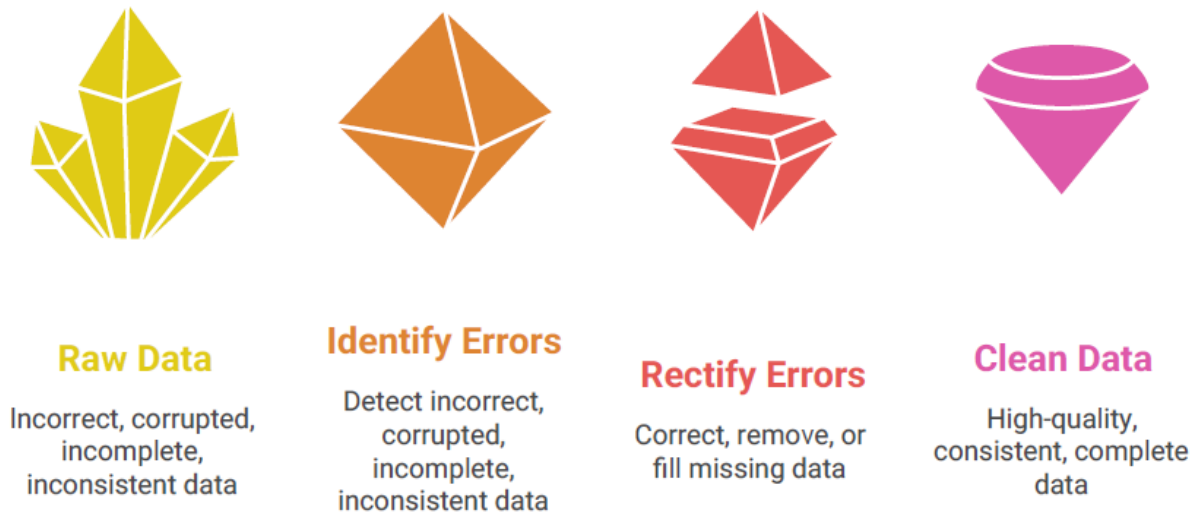
Ch-2.3: Feature Understanding



1. **df["feature"].value_counts()** :
Used to count unique values in a categorical feature.
2. **df["feature"].unique()** :
Used to display unique values in a feature.
3. **df["feature"].nunique()** :
Used to count number of unique values.
4. **df.corr()** :
Used to compute correlation between numerical features.
5. **df.corr()["target"]** :
Used to check correlation of all numerical features with the target variable

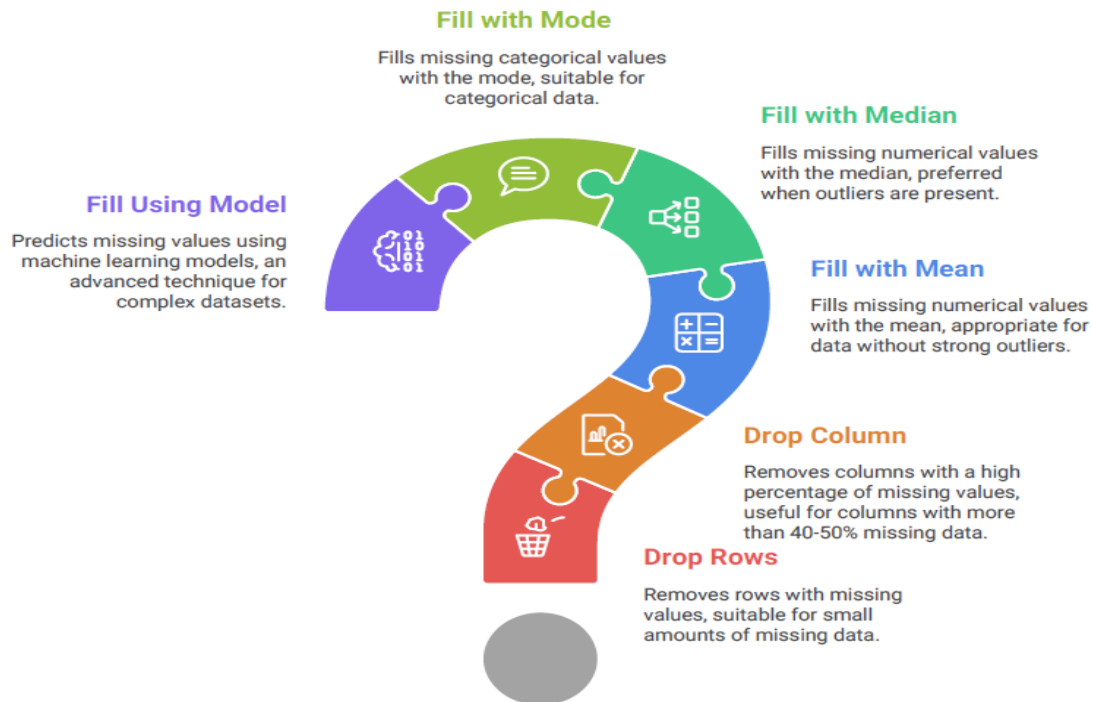
Ch-3: Data Cleaning

Data Cleaning is the process of fixing or removing incorrect, corrupted, incomplete, or inconsistent data to improve data quality before modeling.



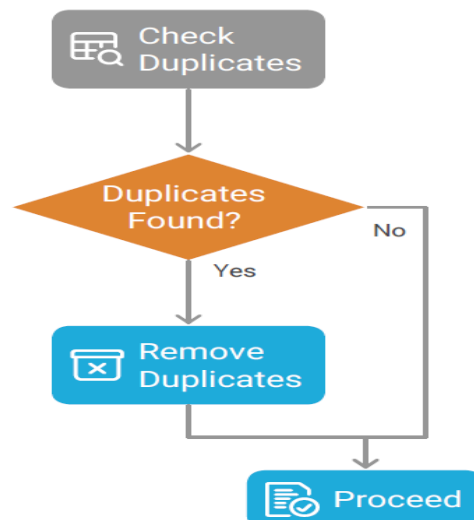
Ch-3.1: Missing Values

How to handle missing values in the dataset?



- **Check Missing Values:**
 1. **`df.isnull().sum()`** :
Used to count missing values in each column.
 2. **`df.isnull().mean() * 100`** :
Used to calculate percentage of missing values.
- **Handling Options**
 - Drop Rows:
`df.dropna()` : Removes rows containing missing values.
 - Drop Column:
`df.drop(columns=["column_name"])` : Removes a column completely.
If missing values are greater than 40%–50%, dropping the column is often considered.
 - Fill with forward fill
`df.ffill()` or **`df.fillna(method='ffill')`**: Propagates the last valid observation forward.
 - Fill with backward fill
`df.bfill()` or **`df.fillna(method='bfill')`**: Propagates the next valid observation backward.
 - Fill with Mean:
`df["column"].fillna(df["column"].mean())` : Used for numerical data without strong outliers.
 - Fill with Median:
`df["column"].fillna(df["column"].median())` : Preferred when outliers are present.
 - Fill with Mode:
`df["column"].fillna(df["column"].mode()[0])` : Used for categorical variables.
 - Fill Using Model (Advanced):
Predict missing values using machine learning models trained on non-missing data.

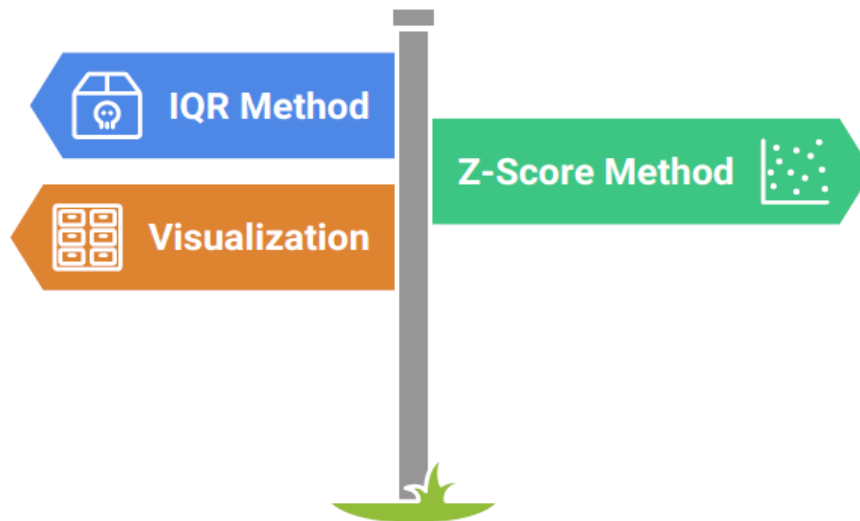
Ch-3.2: Duplicates



- **Check Duplicates**
`df.duplicated().sum()` : Counts duplicate rows.
- **Remove Duplicates**
`df.drop_duplicates()` : Removes duplicate rows from the dataset.

Ch-3.3: Outliers

How to detect and handle outliers in the dataset?



Outliers are extreme values that differ significantly from other observations.

1. IQR Method:

Detect outliers using Interquartile Range.

$Q1 = df["column"].quantile(0.25)$

$Q3 = df["column"].quantile(0.75)$

$IQR = Q3 - Q1$

➤ **Outlier condition:**

$df[(df["column"] < Q1 - 1.5 * IQR) \mid (df["column"] > Q3 + 1.5 * IQR)]$

2. Z-Score Method

Detect outliers based on standard deviation.

Code:

```
from scipy import stats
import numpy as np
z = np.abs(stats.zscore(df["column"]))
df[z > 3]
```

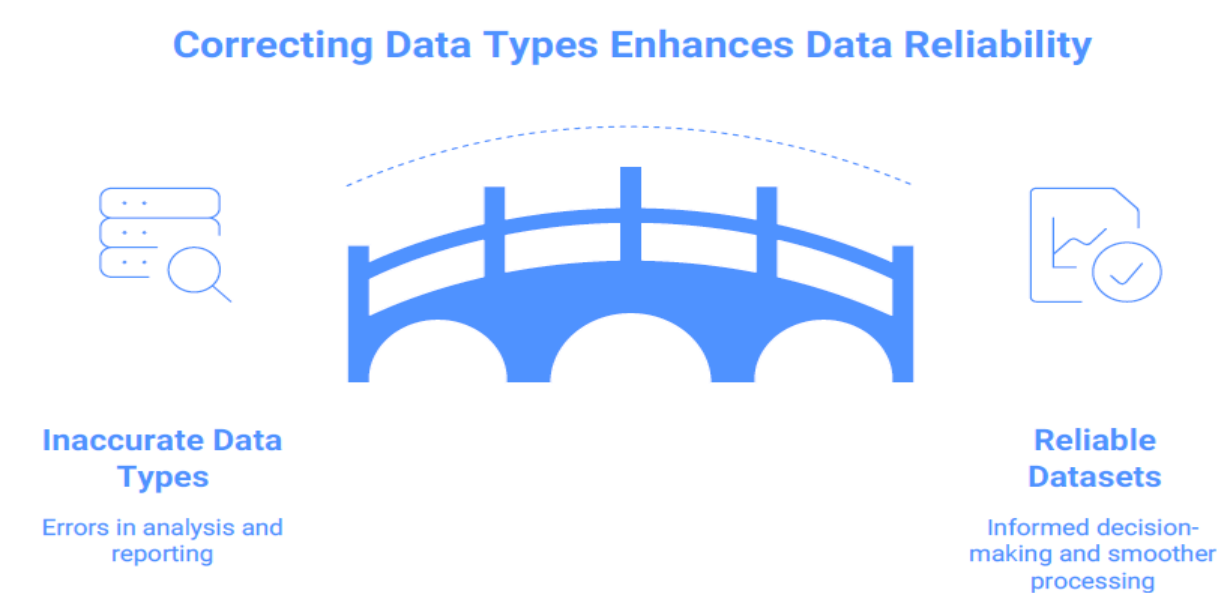
3. Visualization

`df.boxplot(column="column")` : Used to visually detect outliers.

Important:

Outliers may represent real business behavior, so removal should be done carefully.

Ch-3.4: Data Type Correction



Sometimes data types are incorrect.

Example problems:

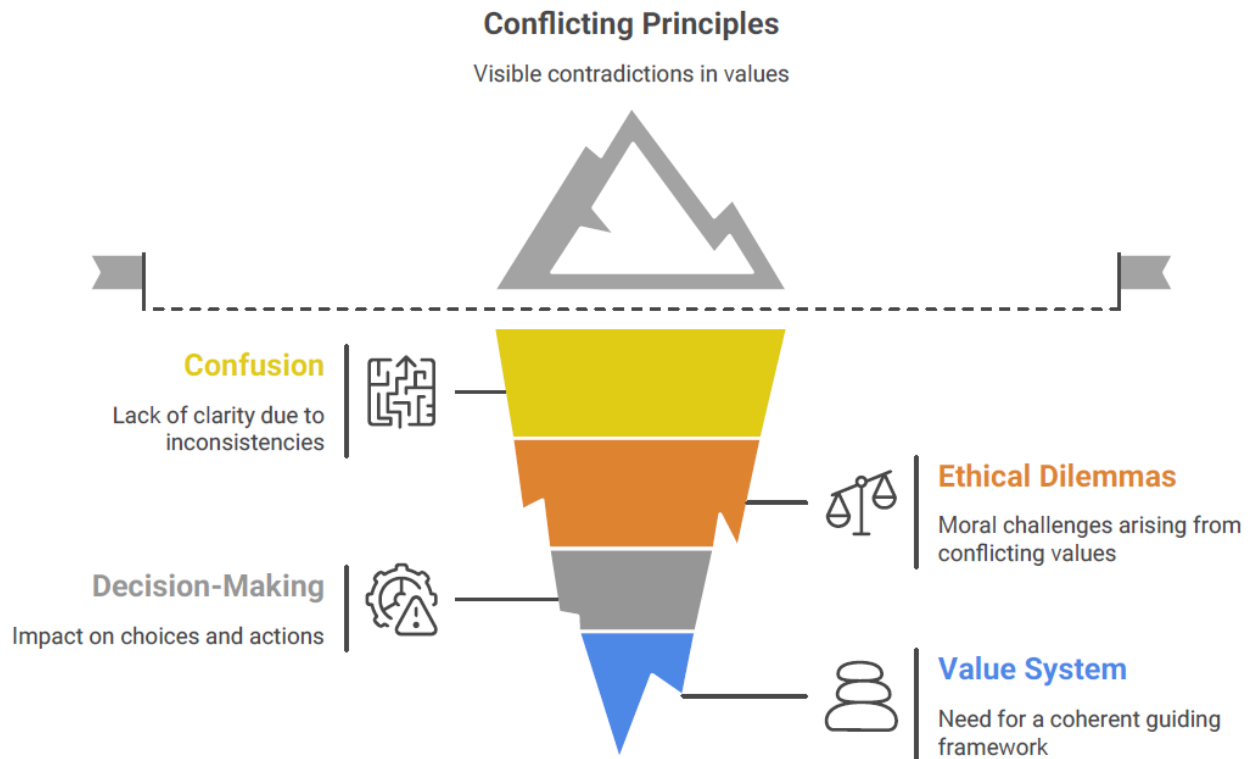
- Age stored as object
- Date stored as string

Convert to Proper Type:

- `df["Age"] = df["Age"].astype(int)` :**
Converts column to integer type.
- `df["Date"] = pd.to_datetime(df["Date"])` :**
Converts string to datetime format.
- `df["column"] = df["column"].astype(float)` :**
Converts to float type.

Ch-3.5: Inconsistent Values

Inconsistent Values: Unveiling the Hidden Depths



Example:

Male / male / M / m

Fix Using Cleaning:

1. `df["Gender"] = df["Gender"].str.lower() :`

Converts all values to lowercase.

2. `df["Gender"].replace({"m": "male"}) :`

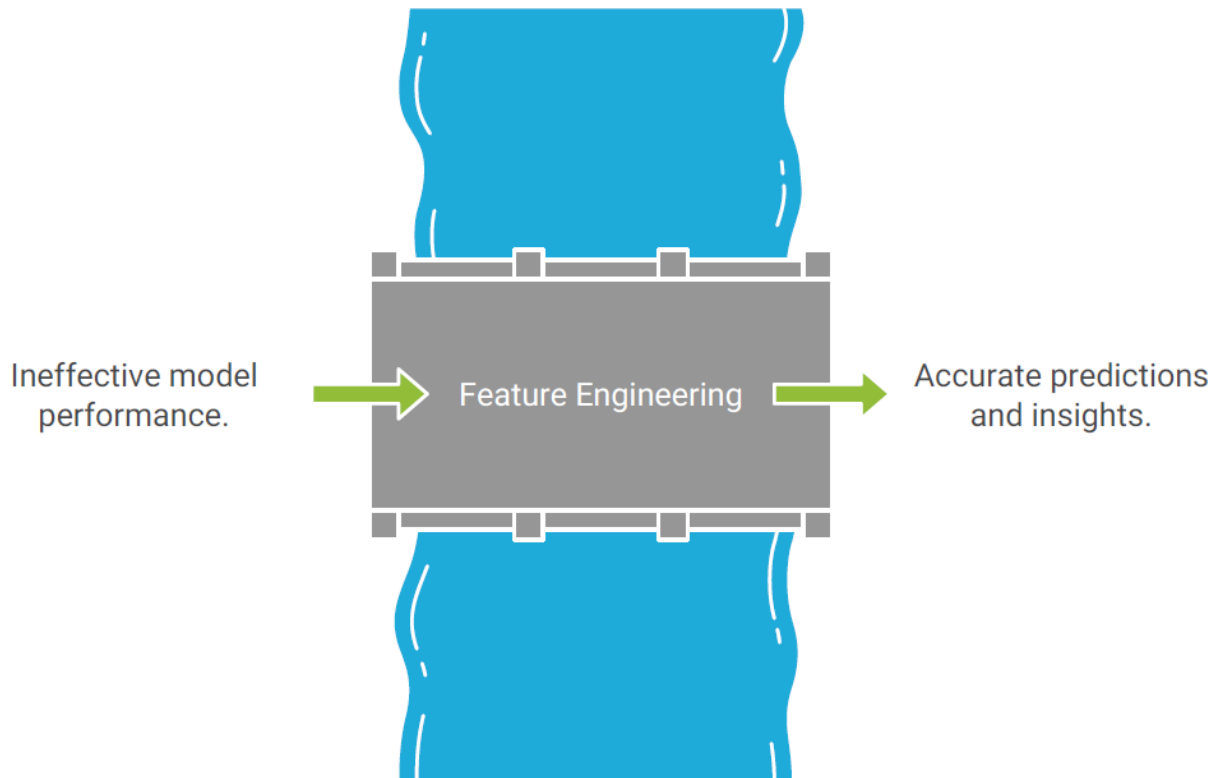
Replaces inconsistent labels with standard format.

This ensures uniform categorical values across the dataset.

Ch-4: Feature Engineering

Feature Engineering is the process of transforming raw data into meaningful features that improve model performance.

Feature Engineering transforms raw data into actionable intelligence.



Ch-4.1: Encoding Categorical Variables

1. Label Encoding

Used when categories have an order (ordinal data).

Example: Low < Medium < High

Code:

```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
df["Education_Level"] = le.fit_transform(df["Education_Level"])
```

Example:

Low → 0

Medium → 1

High → 2

2. One-Hot Encoding

Used when there is no order between categories (nominal data).

```
df = pd.get_dummies(df, columns=["City"], drop_first=True)
```

If City = Mumbai, Delhi, Chennai

It creates:

City_Delhi

City_Mumbai

(No ranking is introduced.)

3. Target Encoding (Advanced)

Replace each category with the mean of target variable for that category.

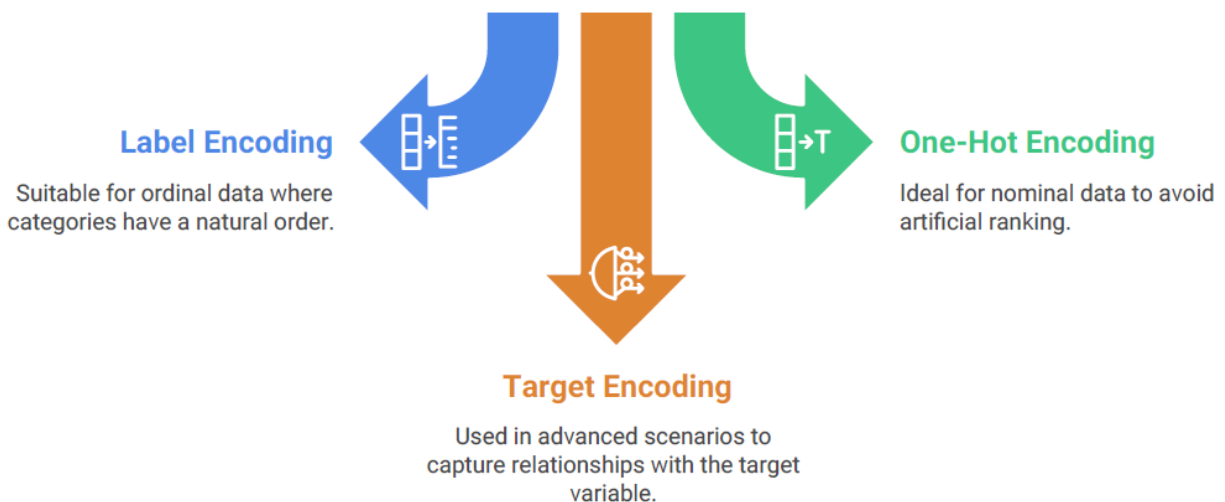
Example:

```
target_mean = df.groupby("City")["Target"].mean()
```

```
df["City_encoded"] = df["City"].map(target_mean)
```

Each city is replaced with its average target value.

Which encoding technique should be used for categorical variables?



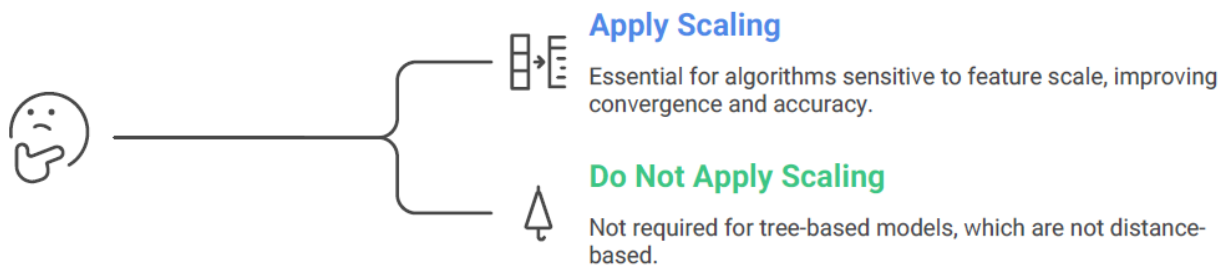
Ch-4.2: Feature Scaling

Scaling ensures numerical features are on similar scale.

Important for:

- KNN
- SVM
- Logistic Regression
- Neural Networks

Should feature scaling be applied to the data?



1. StandardScaler

Mean = 0, Standard Deviation = 1

Code:

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
df[["Age", "Salary"]] = scaler.fit_transform(df[["Age", "Salary"]])
```

2. MinMaxScaler

Scales values between 0 and 1.

Code:

```
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
df[["Age", "Salary"]] = scaler.fit_transform(df[["Age", "Salary"]])
```

Tree-Based Models:

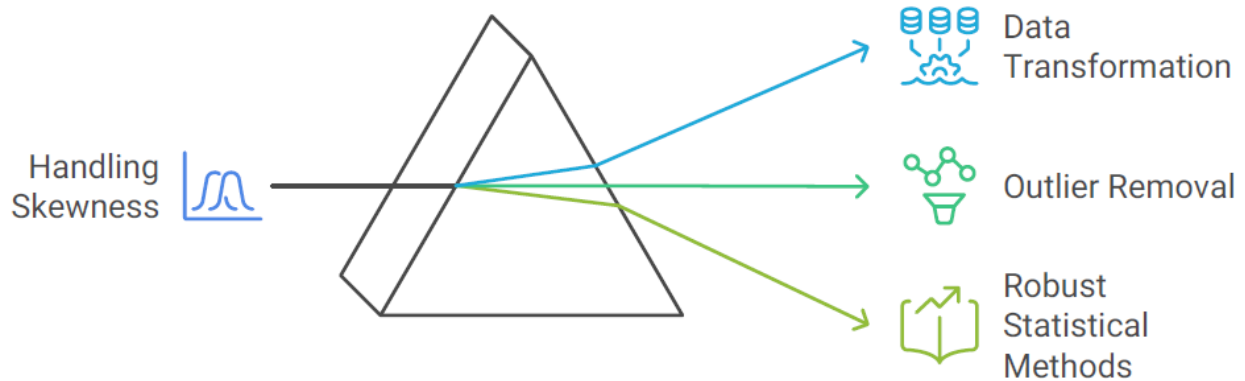
Models like:

- Random Forest
- Decision Tree
- XGBoost

Do not require scaling because they are not distance-based.

Ch-4.3: Handling Skewness

Unveiling the Dimensions of Handling Skewness



If numerical feature is highly skewed:

1. Log Transformation

```
import numpy as np
```

```
df["Salary"] = np.log(df["Salary"])
```

Used when distribution is right-skewed.

2. Box-Cox Transformation

```
from scipy.stats import boxcox
```

```
df["Salary"], _ = boxcox(df["Salary"])
```

Used to make distribution more normal.

Ch-4.4: Feature Creation

Creating new meaningful variables from existing data.

- **Example 1: Age from DOB**

```
df["DOB"] = pd.to_datetime(df["DOB"])
```

```
df["Age"] = 2026 - df["DOB"].dt.year
```

- **Example 2: Profit Calculation**

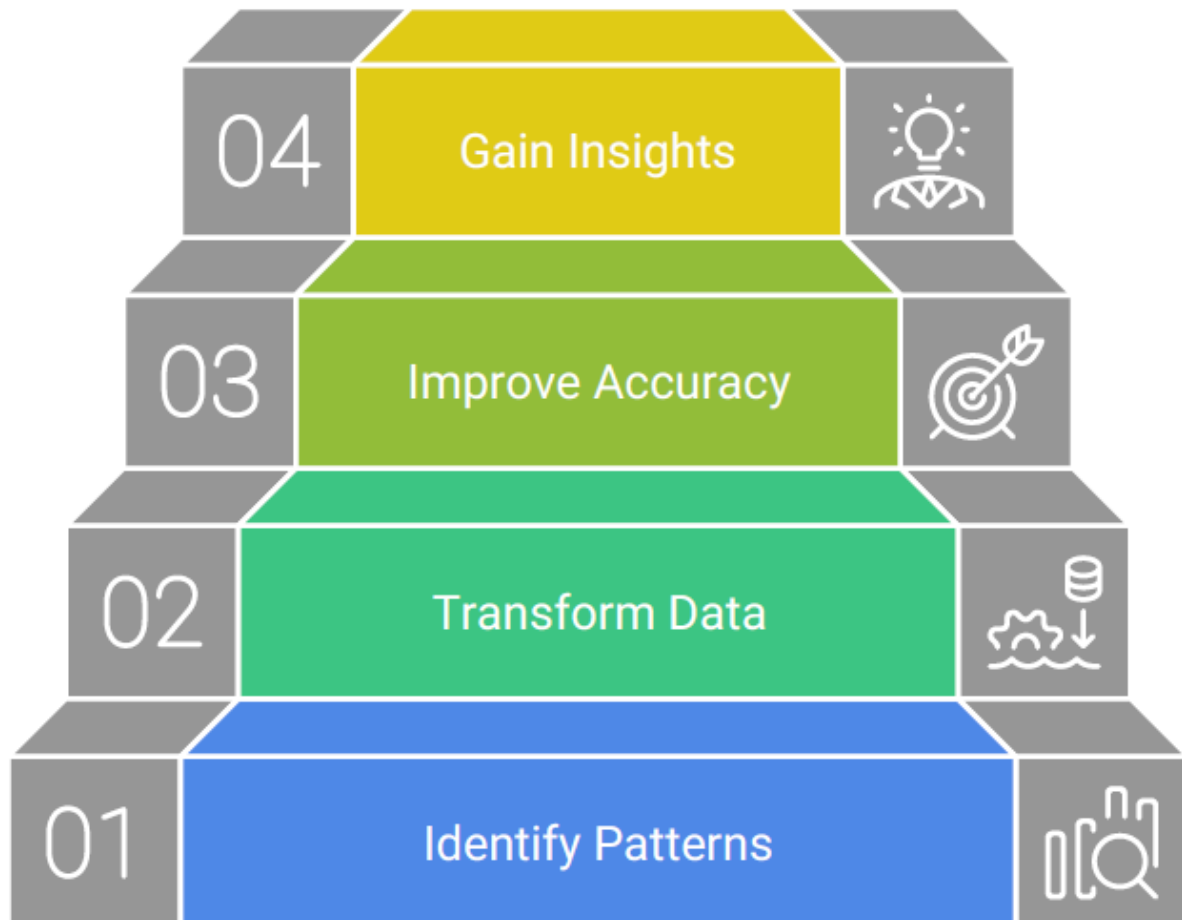
```
df["Profit"] = df["Revenue"] - df["Cost"]
```

- **Example 3: Extract Date Components**

```
df["Year"] = df["Date"].dt.year
```

```
df["Month"] = df["Date"].dt.month  
df["Day"] = df["Date"].dt.day
```

Achieving Enhanced Predictive Power



ch-4.5: Feature Selection

Selecting the most relevant features improves performance and reduces overfitting

1. Filter Method

- **Correlation**

```
df.corr()["Target"].sort_values(ascending=False)
```

Remove highly correlated features (> 0.8).

- **Chi-Square (for categorical input & classification target)**

```
from sklearn.feature_selection import chi2
```

Mutual Information

```
from sklearn.feature_selection import mutual_info_classif
```

2. Wrapper Method

- **Recursive Feature Elimination (RFE)**

```
from sklearn.feature_selection import RFE
from sklearn.linear_model import LogisticRegression
model = LogisticRegression()
rfe = RFE(model, n_features_to_select=5)
rfe.fit(X, y)
```

3. Embedded Method

- **Lasso (L1 Regularization)**

```
from sklearn.linear_model import Lasso
model = Lasso(alpha=0.1)
model.fit(X, y)
Features with zero coefficients are removed.
```

- **Tree-Based Feature Importance**

```
from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier()
model.fit(X, y)
importance = model.feature_importances_
```

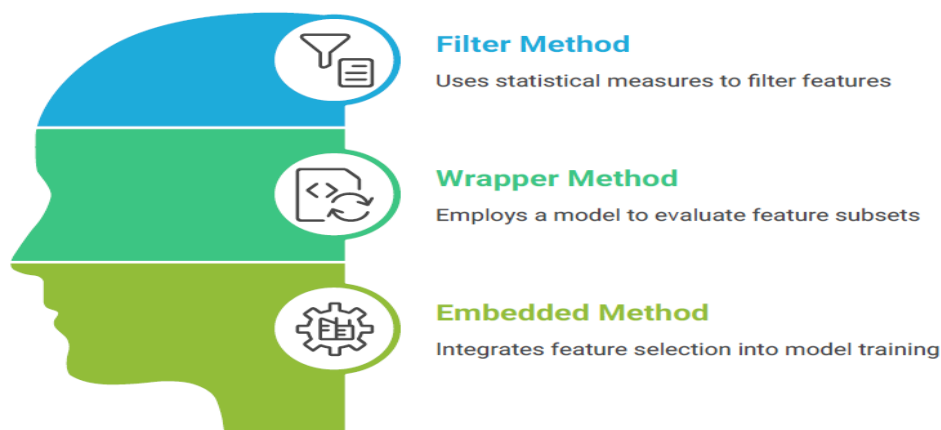
- **Remove:**

Low Variance Features:

```
from sklearn.feature_selection import VarianceThreshold
selector = VarianceThreshold(threshold=0.01)
X = selector.fit_transform(X)
```

Highly Correlated Features (> 0.8 correlation).

Enhancing Model Performance with Feature Selection



Ch-4.6: Train-Test Split

Train-Test Split is the process of dividing the dataset into two parts so that the model can be trained on one portion of the data and evaluated on unseen data.

Balancing Data for Model Development



Usually:

Training Set → 80% of the data (used to train the model)

Test Set → 20% of the data (used to evaluate model performance)

1. Basic Train-Test Split:

```
from sklearn.model_selection import train_test_split
X = df.drop("target", axis=1)
y = df["target"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

- **test_size=0.2 :**
Used to allocate 20% data for testing.
- **random_state=42 :**
Used to ensure reproducibility of the split.

2. Stratified Train-Test Split (For Imbalanced Data)

Used when class distribution must remain the same in both training and testing datasets.

Code:

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
```



```
test_size=0.2,  
stratify=y,  
random_state=42  
)
```

- **stratify=y :**
Ensures class proportions remain the same in train and test datasets.

Ch-5: Model Selection

Definition: Model selection is the process of choosing the most suitable machine learning algorithm for a problem based on the type of data, problem type, and complexity of the dataset.

Structured Data

Suitable for models like regression and classification, both supervised and unsupervised.

Unstructured Data

Requires models designed for text, image, audio, or video processing.



Which machine learning model should be selected for the problem?

Machine learning models are commonly categorized based on the type of data:

1. Structured Data:
 - I. Supervised Machine learning model:
 - a. Regression
 - b. Classification
 - II. Unsupervised Machine learning model
2. Unstructured Data:
 - I. Text
 - II. Image
 - III. Audio
 - IV. Video

Ch-5.1: Models for Structured Data

Structured data is tabular data organized in rows and columns (like CSV, Excel, SQL tables).

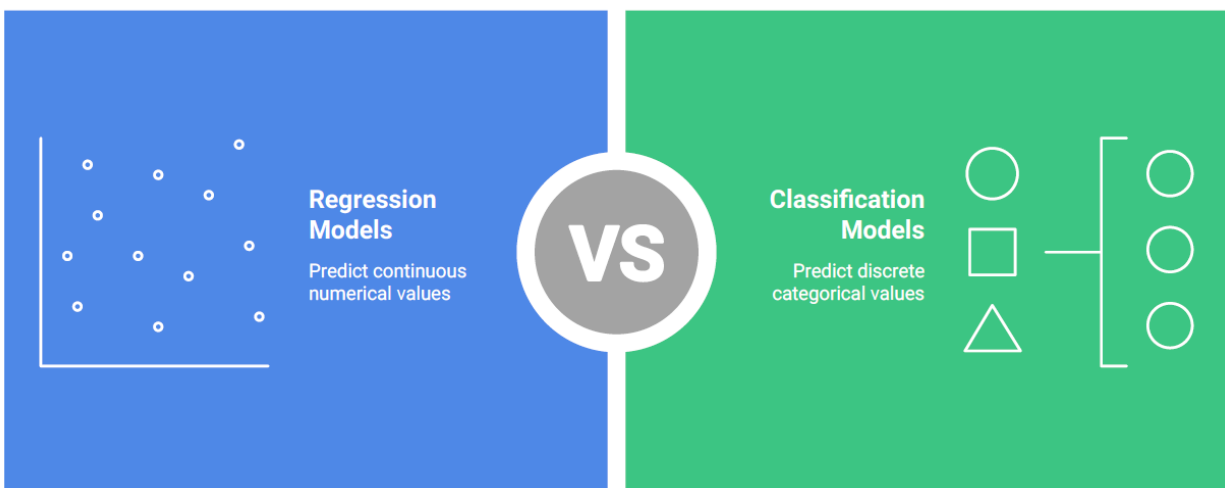
Examples:

Customer data, sales data, banking data.

Ch-5.1.1: Supervised machine learning model

A supervised learning model is a type of machine learning where the algorithm learns from labeled data—meaning each training example comes with an input and a correct output.

Choose the appropriate supervised learning model for your data.



Ch-5.1.1.1 Regression Models (predict numbers)

Regression is used when the output is a continuous numerical value. In simple words it predicts “how much” or “how many”.

Example:

- Predicting house price
- Predicting temperature
- Predicting salary based on experience

1. Linear Regression:

Fits a straight-line relationship between input and output.

Example: Predicting house prices based on size.

Works best when: Relationship is linear and data is simple.

2. Ridge Regression:

Linear regression with L2 regularization (reduces overfitting by shrinking coefficients).

Example: Predicting sales with many correlated features.

Works best when: Many features and multicollinearity exists.

3. Lasso Regression:

Linear regression with L1 regularization (can set some coefficients to zero → feature selection).

Example: Selecting important features in a dataset.

Works best when: You want automatic feature selection.

4. Support Vector Regression (SVR):

Uses margin-based approach to fit data within a threshold.

Example: Predicting stock prices (approximate trend).

Works best when: Data is non-linear and medium-sized.

Ch-5.1.1.2: Classification Models (predict categories)

Classification is used when the output is a category or label. In simple words it predicts “which type” or “which class”.

Example:

- Email → Spam or Not Spam
- Image → Cat or Dog
- Disease → Positive or Negative

1. Logistic Regression:

Estimates probability using a sigmoid function.

Example: Spam detection (spam / not spam).

Works best when: Binary classification and linear boundaries.

2. Decision Tree:

Splits data into branches based on conditions.

Example: Loan approval decision system.

Works best when: Need interpretability and rule-based logic.

3. Random Forest:

Combines multiple decision trees (ensemble learning).

Example: Customer churn prediction.

Works best when: Want high accuracy and less overfitting.

4. Support Vector Machine (SVM):

Finds the best boundary (hyperplane) between classes.

Example: Image classification.

Works best when: High-dimensional data and clear margins.

5. K-Nearest Neighbors (KNN):

Classifies based on nearest data points.

Example: Recommender systems (similar users).

Works best when: Small datasets and clear similarity patterns.

6. Naive Bayes:

Uses probability with assumption of feature independence.

Example: Email spam filtering.

Works best when: Text data and large datasets.

Ch-5.1.1.3: Models Used for both Classification & Regression

1. Decision Tree:

Can predict categories or numbers.

Example: House price (regression) OR loan approval (classification).

Works best when: Need flexibility + interpretability.

2. Random Forest:

Ensemble of trees for both tasks.

Example: Price prediction or fraud detection.

Works best when: Want robust and accurate results.

3. Gradient Boosting:

Builds models sequentially to correct previous errors.

Example: Credit scoring, ranking problems.

Works best when: Need high performance on complex data.

4. K-Nearest Neighbors (KNN):

Can predict class (majority vote) or value (average).

Example: Price prediction OR classification.

Works best when: Small datasets and local patterns.

5. **Support Vector Machine (SVM / SVR combined):**

Works for both classification (SVM) and regression (SVR).

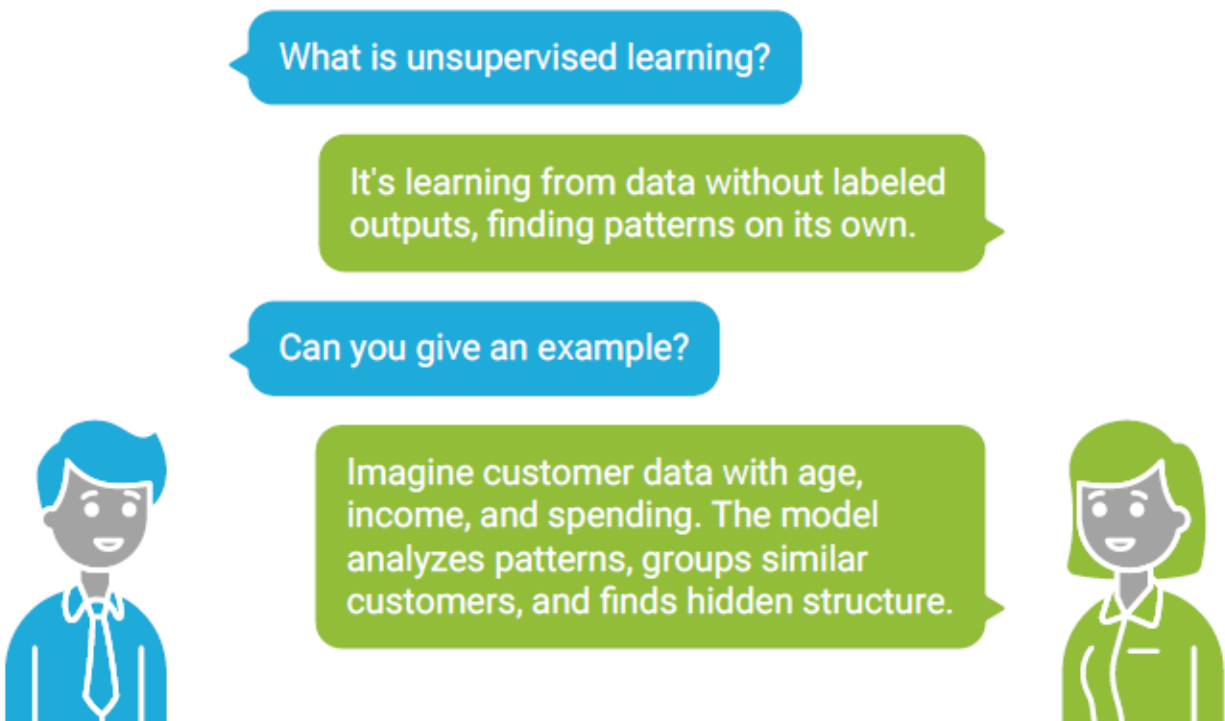
Example: Image classification or price prediction.

Works best when: Data is high-dimensional.

Ch-5.1.2: unsupervised learning (Clustering Models)

Unsupervised learning is a type of machine learning where the model learns from data without labeled outputs. In simple words the model is given data without answers and must find patterns on its own.

Unsupervised Learning: Clustering Models



Example:

I. Customer data with age, income, spending

- II. Model analyzes patterns
- III. It tries to group or organize the data
- IV. Finds hidden structure
- V. Like grouping similar customers together

1. K-Means Clustering:

Groups data into K clusters based on similarity.

Example: Customer segmentation.

Works best when: Clusters are well-separated and spherical.

2. Hierarchical Clustering:

Builds a tree of clusters (dendrogram).

Example: Grouping similar documents.

Works best when: Need hierarchy of clusters.

3. Gaussian Mixture Model (GMM)

Assumes data comes from multiple Gaussian distributions.

Example: Identifying overlapping customer segments.

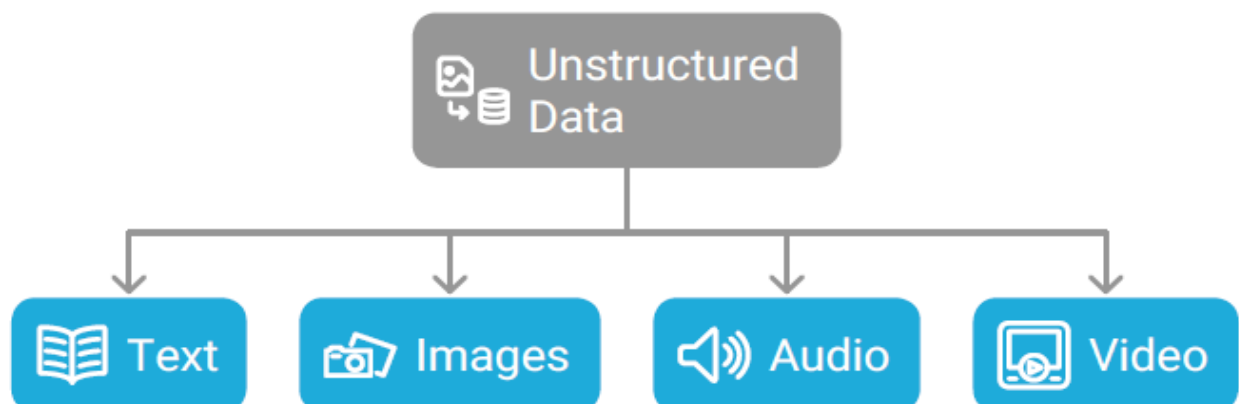
Works best when: Clusters are overlapping or soft boundaries.

Ch-5.2: Models for Unstructured Data

Unstructured data is data that does not follow a tabular format.

Examples:

- Text
- Images
- Audio
- Video



Ch-5.2.1: Natural Language Processing (Text Data)

Used for analyzing text data.

Common models:

➤ Naive Bayes

Used for text classification.

Example:

Spam detection, sentiment analysis.

```
from sklearn.naive_bayes import MultinomialNB
```

Works well for:

Text classification tasks.

Ch-5.2.2: TF-IDF + Machine Learning

Text is converted to numerical features using TF-IDF and then used with models like Logistic Regression or SVM.

Example:

Document classification.

Ch-5.2.3: Transformers / Deep Learning

Modern NLP models.

Examples:

➤ BERT

➤ GPT

➤ RoBERTa

Used for:

Chatbots, translation, question answering.

Ch-5.2.4: Image Data

Used for analyzing images.

Common models:

Convolutional Neural Networks (CNN)

Example architectures:

- ResNet
- VGG
- EfficientNet

Used for:

- Image classification
- Object detection
- Face recognition.

Ch-5.2.5: Audio Data

Used for speech or sound analysis.

Common models:

- CNN
- RNN
- Transformers

Applications:

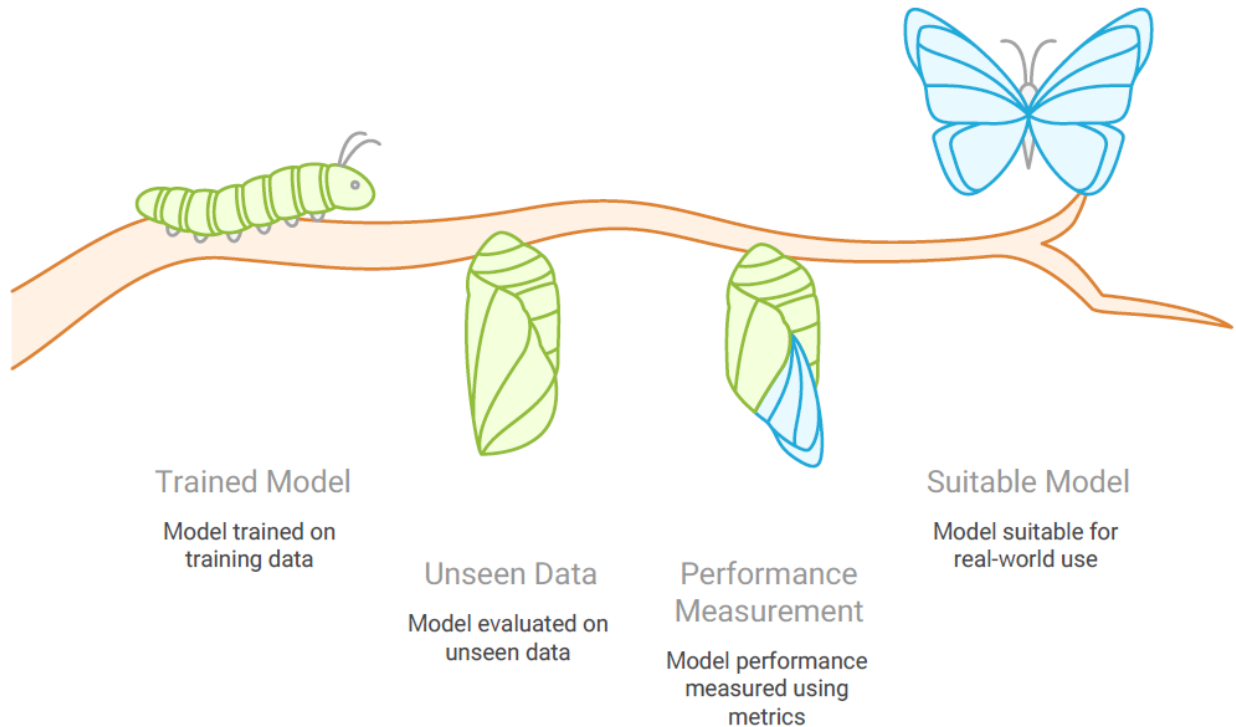
Speech recognition, speaker identification.

These models are selected based on **data type, dataset size, complexity, and problem type (classification, regression, clustering)**.

Ch-6: Model evaluation

Model evaluation is the process of measuring how well a trained machine learning model performs on unseen data. It helps determine whether the model generalizes well and whether it is suitable for real-world use.

Model Evaluation Process



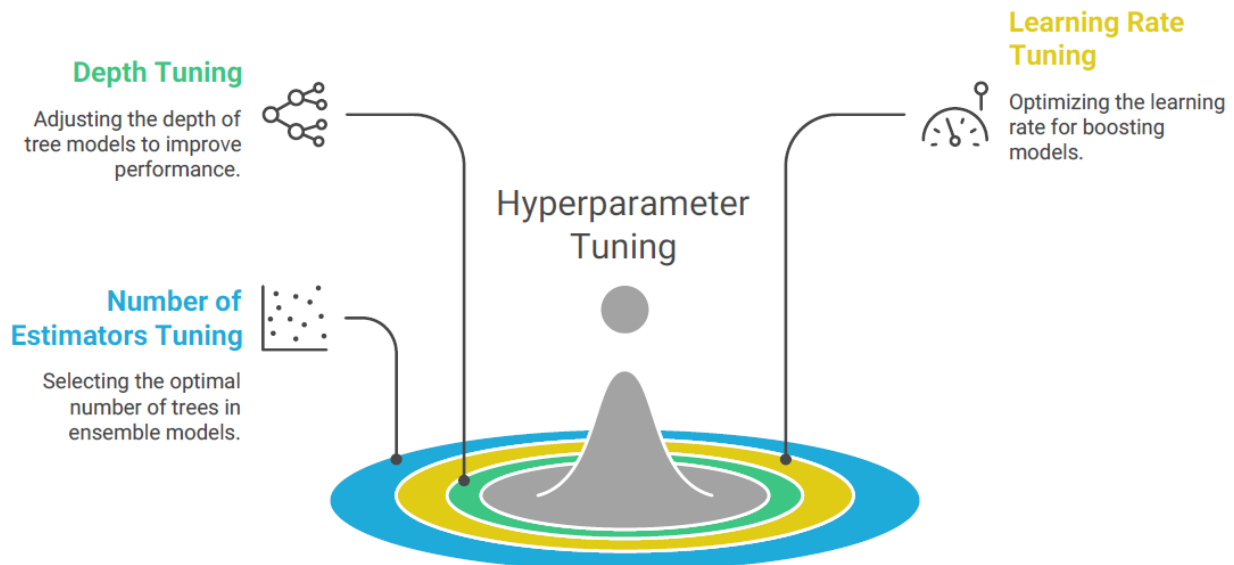
Ch-6.1: Hyperparameter Tuning

Hyperparameters are configuration values set before training the model. Tuning them improves model performance.

Common hyperparameters to tune:

- **Depth** (for tree models)
- **Learning Rate** (for boosting models)
- **Number of Estimators** (number of trees in ensemble models)

Hyperparameter Tuning Strategies



1. GridSearchCV

Searches all possible combinations of hyperparameters.

Code:

```
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestClassifier
params = {
    "n_estimators": [100, 200],
    "max_depth": [5, 10, 15]
}
grid = GridSearchCV(RandomForestClassifier(), params, cv=5)
grid.fit(X_train, y_train)
grid.best_params_
```

Used for exhaustive search of parameter combinations.

2. RandomizedSearchCV

Tests random combinations instead of all possibilities.

Code:

```
from sklearn.model_selection import RandomizedSearchCV
random_search = RandomizedSearchCV(
    RandomForestClassifier(),
    params,
```

```
n_iter=5,  
cv=5  
)  
random_search.fit(X_train, y_train)
```

Used when parameter space is large.

3. Optuna (Advanced)

A modern library for automatic hyperparameter optimization.

Used for:

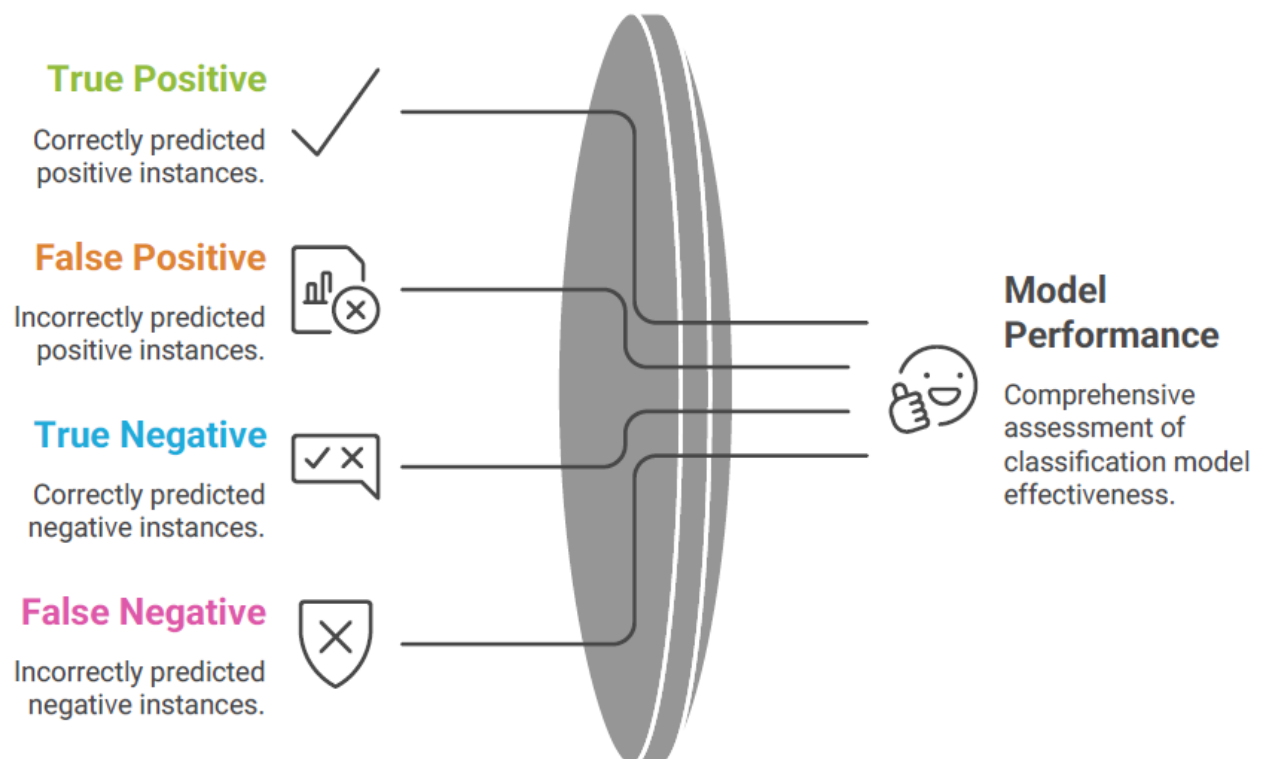
- Deep learning
- Large machine learning models.

Ch-6.2: Deep Evaluation

After training the model, predictions are evaluated using performance metrics.

Confusion Matrix: A confusion matrix shows how well the classification model predicts each class.

Evaluating Classification Accuracy

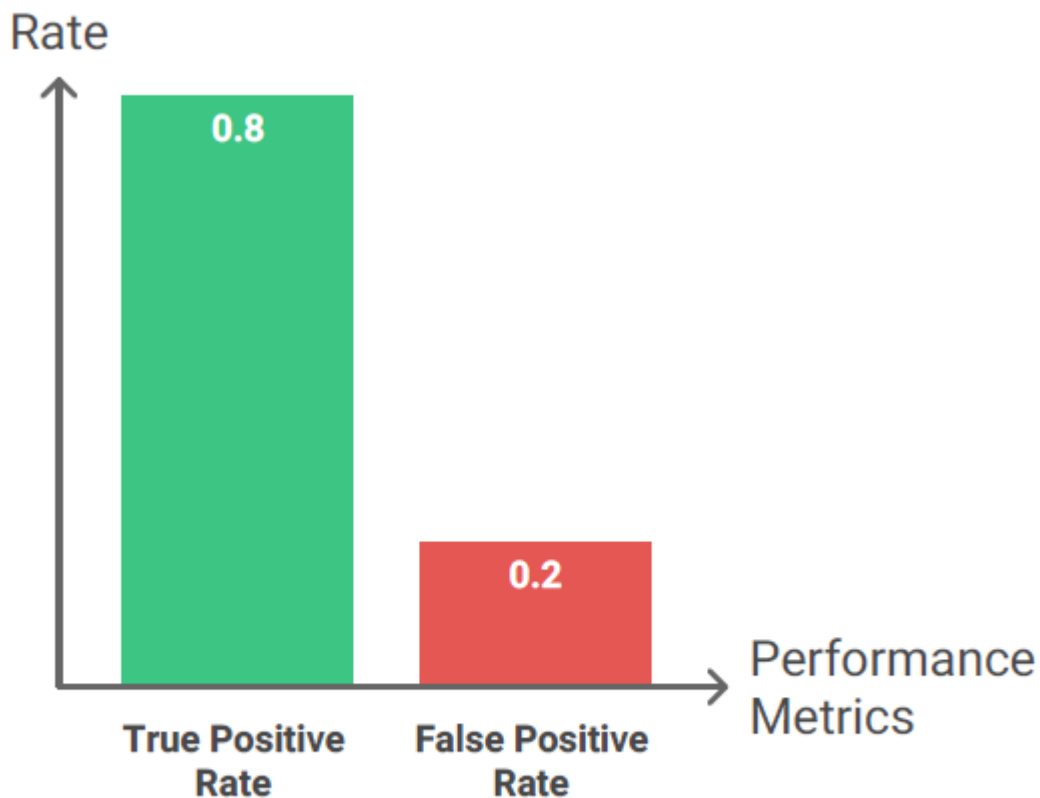


Code:

```
from sklearn.metrics import confusion_matrix  
y_pred = model.predict(X_test)  
confusion_matrix(y_test, y_pred)
```

Confusion Matrix Components

- **True Positive (TP):** Model correctly predicts positive class.
- **False Positive (FP):** Model predicts positive but actual class is negative.
- **True Negative (TN):** Model correctly predicts negative class.
- **False Negative (FN):** Model predicts negative but actual class is positive.

Ch-6.3: ROC Curve**ROC Curve Performance Metrics**

ROC curve shows performance of a classifier across different thresholds.

```
from sklearn.metrics import roc_curve
```

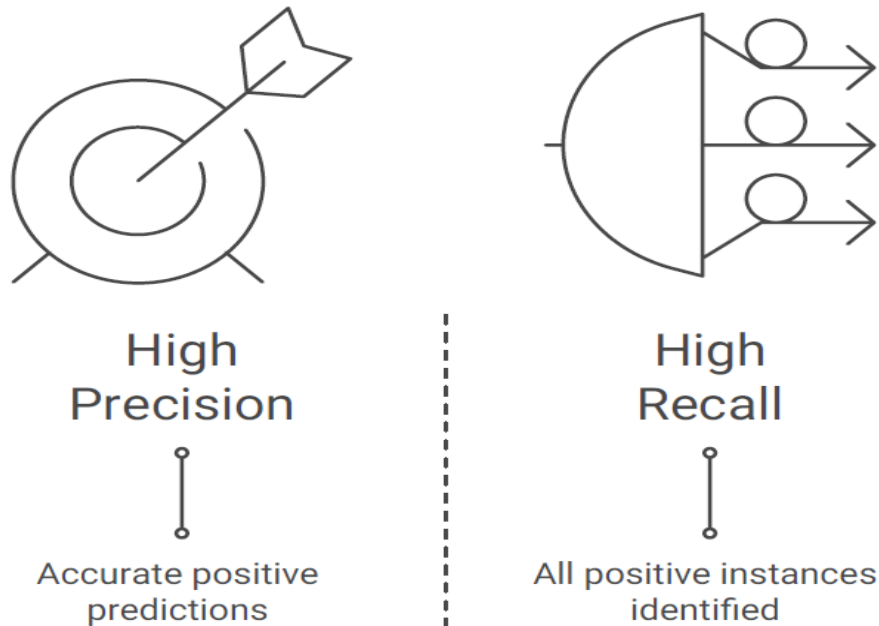
It plots:

- True Positive Rate
- False Positive Rate

Used to evaluate classification performance.

Ch-6.4: Precision–Recall Curve

Precision vs Recall



Used especially when dataset is imbalanced.

from `sklearn.metrics` import `precision_recall_curve`

It plots relationship between:

- Precision
- Recall

Ch-6.5: Threshold Tuning

Classification models usually use 0.5 probability threshold.

Example:

If prediction probability $> 0.5 \rightarrow \text{class} = 1$.

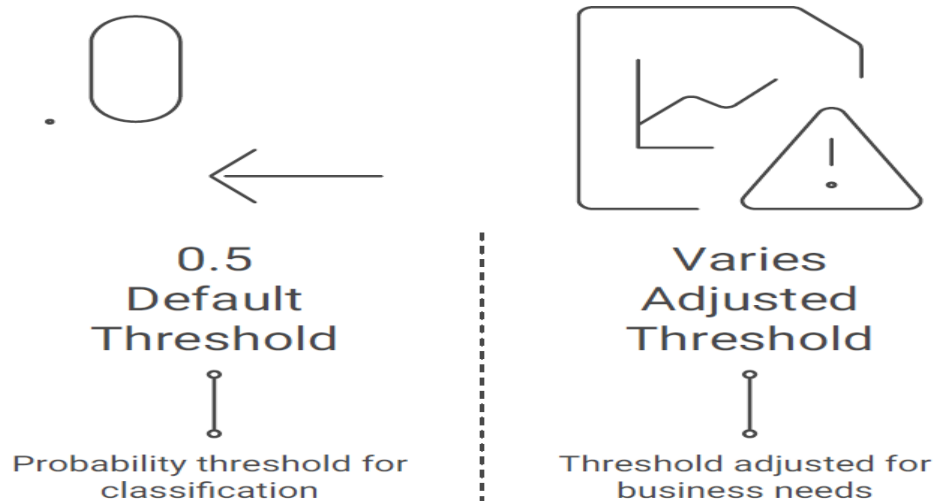
However, the threshold can be adjusted depending on business needs.

Example:

Fraud detection $\rightarrow$ lower threshold

Medical diagnosis $\rightarrow$ minimize false negatives.

Classification Threshold



Ch-6.6: Feature Importance

Feature importance helps identify which variables contribute most to model predictions.

`model.feature_importances`

This shows which features influence the model the most.

Example question:

Which features drive customer churn?

Understanding Feature Importance



Identification of Key Variables

Pinpoints the most influential features



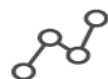
Model Performance Enhancement

Improves accuracy by focusing on relevant features



Business Insights

Provides valuable information for decision-making



Example Application

Illustrates how feature importance can be used

Ch-6.7: SHAP Values (Advanced)

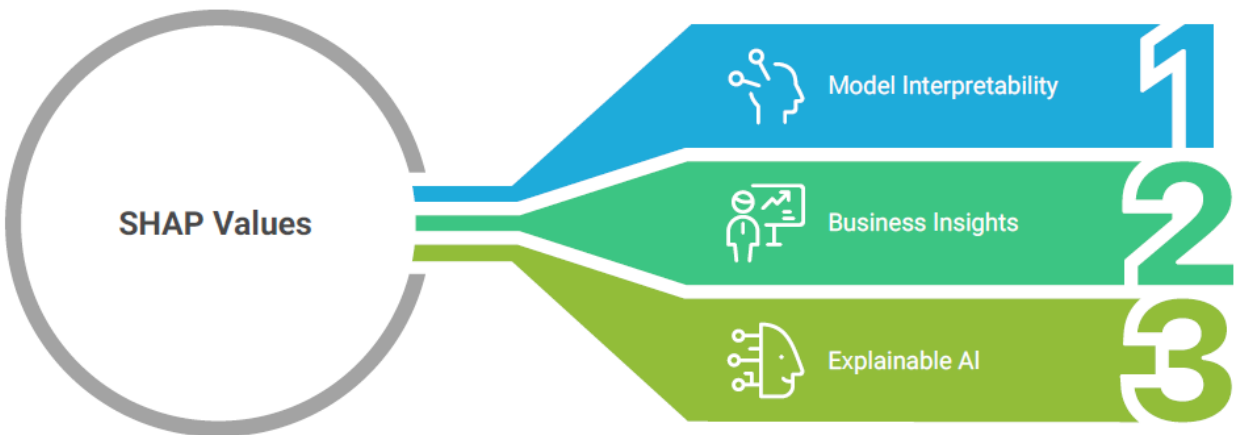
SHAP explains how each feature impacts individual predictions.

Used for:

- Model interpretability
- Business insights
- Explainable AI

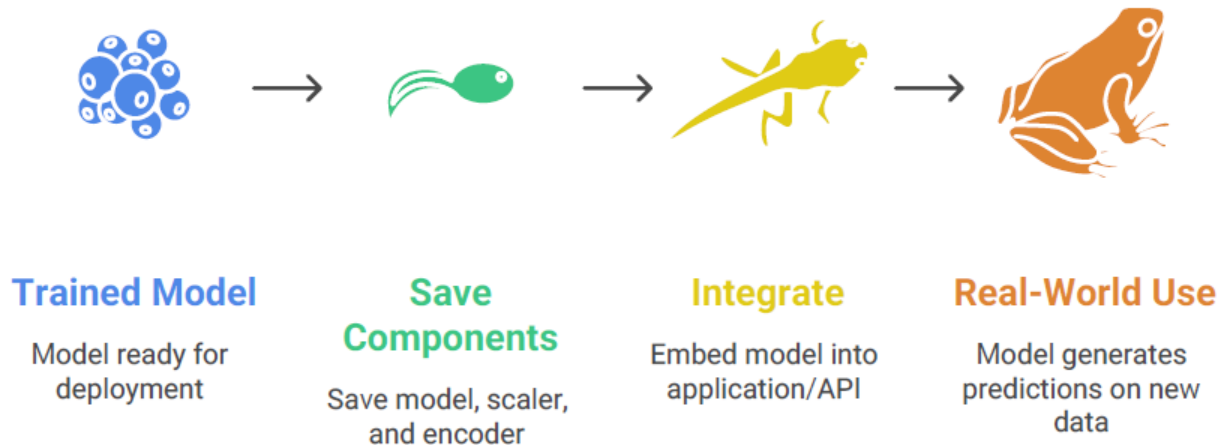
This helps stakeholders understand why the model makes certain predictions, which is often critical for business decisions.

Unveiling the Power of SHAP Values



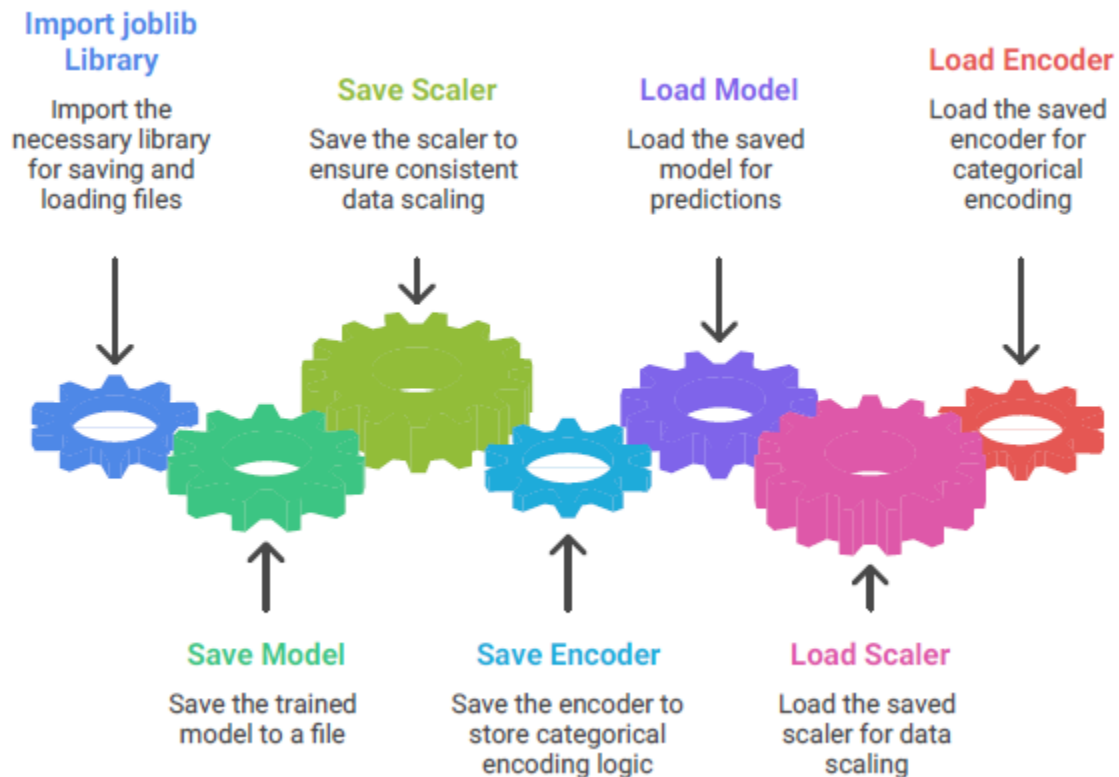
Ch-7: Deployment

Deployment is the process of making a trained machine learning model available for real-world use so it can generate predictions on new data. This involves saving the trained model and integrating it into an application or API.



Ch-7.1: Save Model, Scaler, and Encoder

To reuse the trained model later, important components such as the model, scaler, and encoder must be saved.



Commonly used library: **joblib**

- **Save Model**

```
import joblib
```

```
joblib.dump(model, "model.pkl")
```

Saves the trained model to a file.

- **Save Scaler**

```
joblib.dump(scaler, "scaler.pkl")
```

Ensures new data is scaled in the same way as training data.

- **Save Encoder**

```
joblib.dump(encoder, "encoder.pkl")
```

Stores categorical encoding logic.

- **Load Saved Files**

```
model = joblib.load("model.pkl")
```

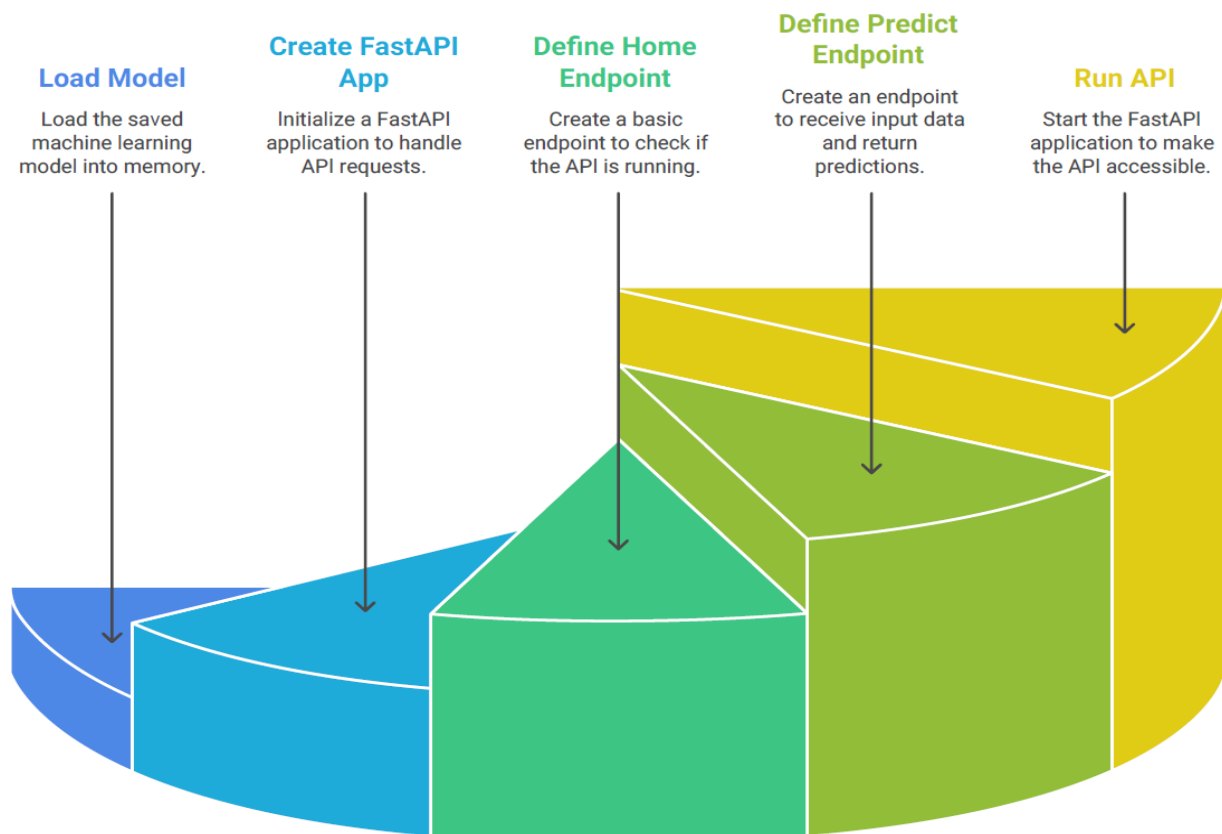
```
scaler = joblib.load("scaler.pkl")
```

```
encoder = joblib.load("encoder.pkl")
```

Loads the saved components for prediction.

Ch-7.2: Build API Using FastAPI

An API allows external applications to send data to the model and receive predictions.



Basic FastAPI Example

```
from fastapi import FastAPI
import joblib
import numpy as np
app = FastAPI()
model = joblib.load("model.pkl")
@app.get("/")
def home():
    return {"message": "Model API is running"}
@app.post("/predict")
def predict(age: float, salary: float):
    data = np.array([[age, salary]])
    prediction = model.predict(data)
    return {"prediction": int(prediction[0])}
Run API
uvicorn main:app --reload
```

The model can now receive input data and return predictions through an API endpoint.

Ch-8: Important Concept

In this chapter covering some important topic which are used in Machine learning

Ch-8.1: Dimensionality Reduction

Used when the dataset has too many features.

PCA (Principal Component Analysis)

PCA reduces the number of features while keeping most of the information.

Purpose:

- Reduce feature count
- Remove multicollinearity
- Speed up training
- Improve visualization

Example:

```
from sklearn.decomposition import PCA
pca = PCA(n_components=5)
X_pca = pca.fit_transform(X)
```

Here:

Original features → compressed into 5 principal components

Commonly used when:

- Dataset has many correlated features
- Image data
- High dimensional datasets.

Should dimensionality reduction be used?

Reduce Feature Count

PCA can significantly reduce the number of features, speeding up training and improving visualization.



Remove Multicollinearity

PCA helps eliminate correlated features, leading to a more robust model.



Handle High Dimensional Data

PCA is effective for datasets with many features, such as image data.



Improve Visualization

Reducing dimensions can make it easier to visualize complex datasets.

Ch-8.2: Learning Curves

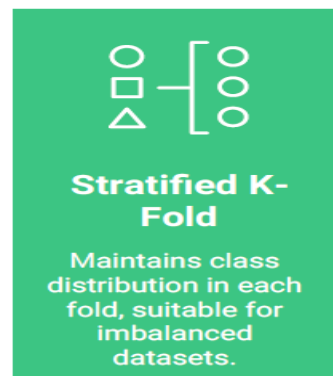
A Learning Curve is a graph that shows how a machine learning model's performance changes with respect to the size of training data.

It typically plots:

- Training Score
- Validation Score
vs
- Number of Training Samples

Learning curves help understand:

- Whether the model is overfitting
- Whether the model is underfitting
- Whether adding more data will improve performance



Key Patterns

1. Overfitting (High Variance)
 - Training score → very high
 - Validation score → low
 - Large gap between both

Meaning:

- Model learns training data too well
- Fails on unseen data

2. Underfitting (High Bias)

- Training score → low
- Validation score → low
- Both are close

Meaning:

- Model is too simple
- Cannot capture patterns

3. Good Fit

- Training score → high
- Validation score → slightly lower
- Small gap

Meaning:

- Model generalizes well

Why It Is Important

- Helps decide model complexity
- Shows if more data is useful
- Guides improvement strategy

Ch-8.3: Handling Imbalanced Data

When one class is much larger than another.

Example:

Fraud detection

99% → Not Fraud

1% → Fraud

- **Techniques:**

- Oversampling
- Increase minority class.

Example: SMOTE

```
from imblearn.over_sampling import SMOTE
```

```
smote = SMOTE()
```

```
X_resampled, y_resampled = smote.fit_resample(X, y)
```

- **Undersampling**

- Reduce majority class.

- Used when dataset is very large.



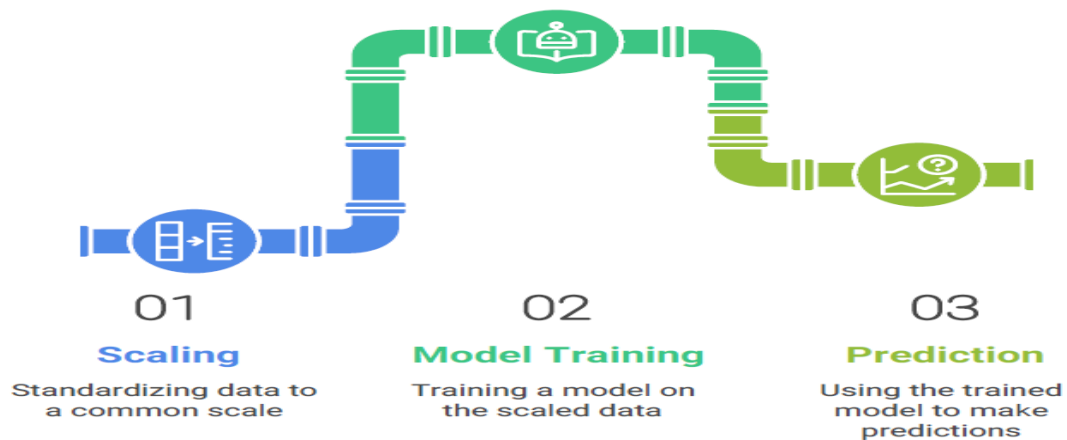
Ch-8.4: Pipelines

Pipelines automate preprocessing + modeling steps.

Example:

Scaling → Model → Prediction

Pipeline Automation in Machine Learning

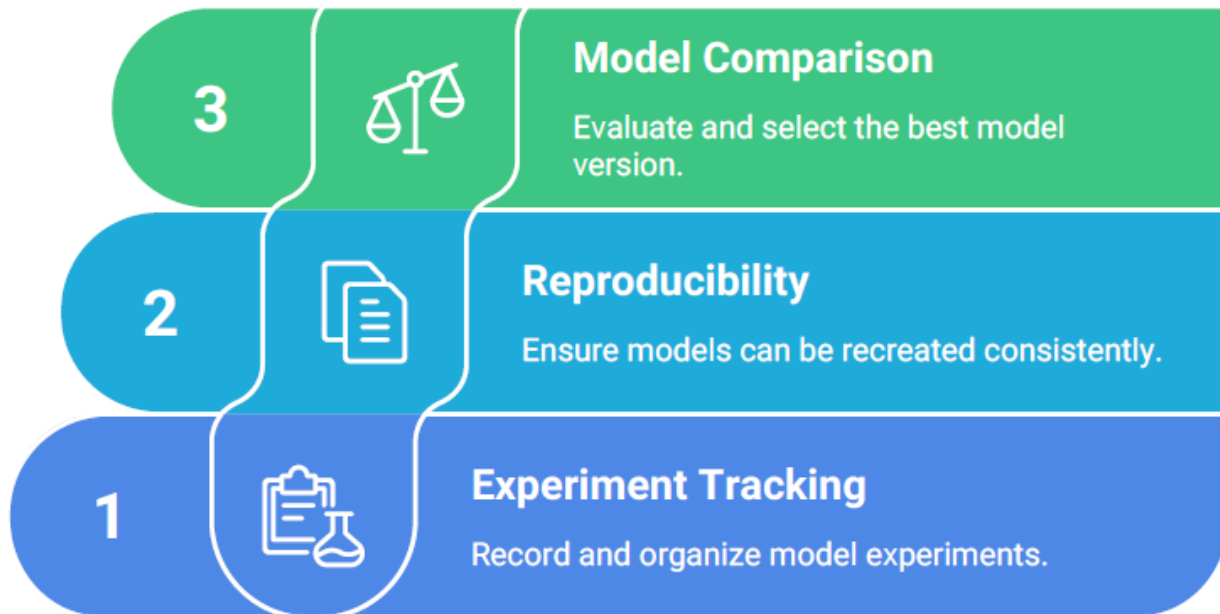


Example:

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("model", LogisticRegression())
])
pipe.fit(X_train, y_train)
```

- **Benefits:**
 - Prevents data leakage
 - Cleaner code
 - Easier deployment.

Ch-8.5: Model Persistence Versioning



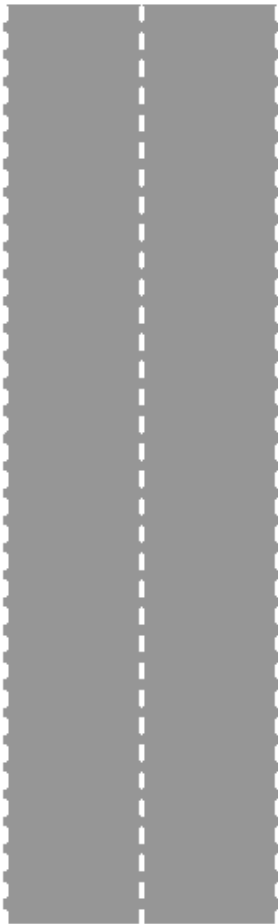
Keeping track of different model versions.

- **Tools commonly used:**
 - MLflow
 - DVC
 - Weights & Biases
- **Purpose:**
 - Experiment tracking
 - Reproducibility
 - Model comparison.

Ch-8.6: Model Monitoring

After deployment, models must be monitored.

- **Important things to track:**
 - Data Drift:** When new data distribution changes.
 - Concept Drift:** When the relationship between input and target changes.
 - Example:** Customer behavior changes over time.



Deploy Model

Model is deployed to production



Track Data Drift

Monitor changes in data distribution



Track Concept Drift

Monitor changes in input-target relationship



Analyze Customer Behavior

Observe changes in customer behavior

Ch-8.7: A/B Testing

Used in production to compare two models.

Example:

Model A → current model

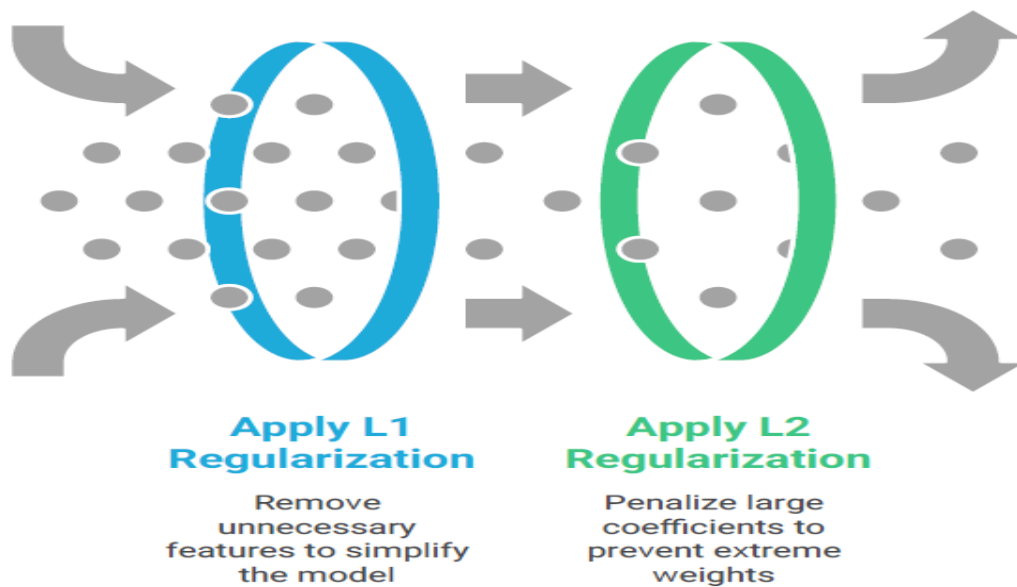
Model B → new model

- **Traffic split:**
 - 50% users → Model A
 - 50% users → Model B
- **Measure:**
 - Conversion rate
 - Accuracy
 - Revenue impact.

Which model performs better in production?

Model A 	Model B 
○ High conversion rate ○ 50% users ○ Conversion rate, Accuracy, Revenue	○ High conversion rate ○ 50% users ○ Conversion rate, Accuracy, Revenue

Ch-8.8: Regularization



Used to reduce overfitting.

- **Types:**

- **L1 Regularization (Lasso):** Removes unnecessary features.
- **L2 Regularization (Ridge):** Penalizes large coefficients.

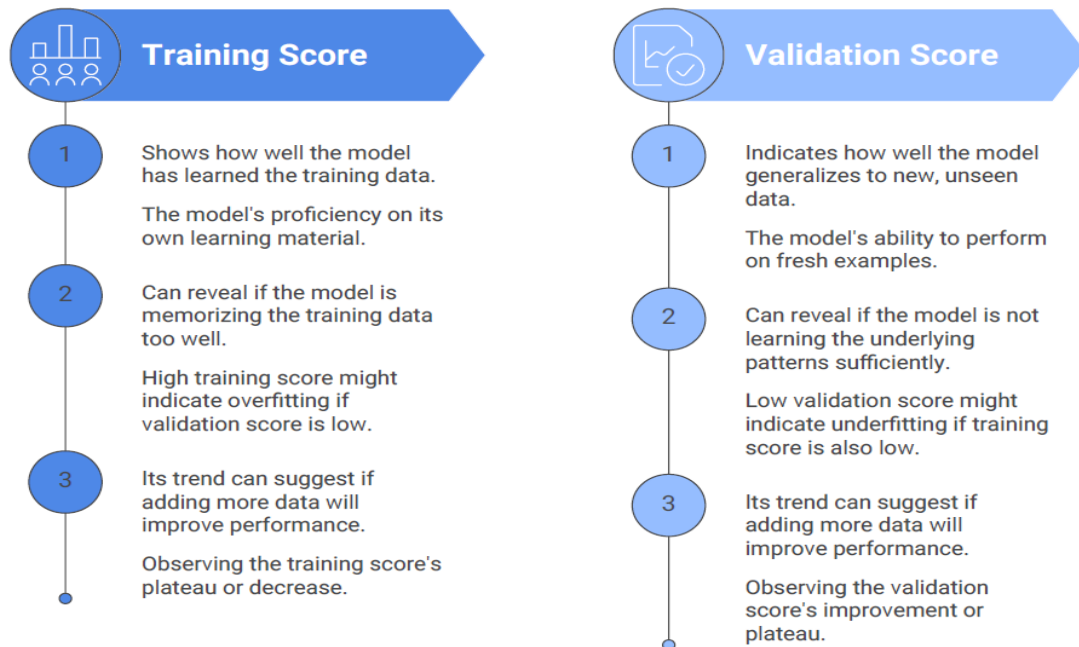
Example:

```
from sklearn.linear_model import Ridge
```

Ch-8.9: Cross Validation

Cross Validation is a technique used to evaluate a machine learning model by training and testing it on multiple different splits of the dataset, instead of relying on a single train-test split.

Which aspect of model performance are you analyzing?



Why It Is Used

- Provides more reliable performance evaluation
- Reduces dependency on one random split
- Helps detect overfitting and underfitting

Ch-8.9.1: K-Fold Cross Validation

The dataset is divided into K equal parts (folds).

Process:

- Train on (K-1) folds
- Test on remaining 1 fold
- Repeat K times with different test folds
- Take average performance

Example:

K = 5 → Model trains 5 times and gives average score.

Example Code

```
from sklearn.model_selection import cross_val_score
```

```

from sklearn.linear_model import LogisticRegression
model = LogisticRegression()
scores = cross_val_score(model, X, y, cv=5)
print(scores)      # Score for each fold
print(scores.mean()) # Average performance

```

Ch-8.9.2: Stratified K-Fold

Used for imbalanced datasets to maintain class distribution in each fold.

```

from sklearn.model_selection import StratifiedKFold
skf = StratifiedKFold(n_splits=5)
scores = cross_val_score(model, X, y, cv=skf)

```

Key Point:

Cross Validation gives a more stable and trustworthy estimate of model performance compared to a single train-test split.

Ch-8.10: Recommendation Systems

- **Used in:**
 - Netflix
 - Amazon
 - YouTube
- **Two main types:**
 1. **Collaborative Filtering:** Based on user behavior.
 2. **Content-Based Filtering:** Based on item features.

